



**EMPLOYEE ID VALIDATOR:
SMART PATTERN RECOGNITION SYSTEM
PROJECT DOCUMENTATION**

Course: Automata Theory and Formal Languages

Group Members:

Anub, Mark Christian T.
Ayson, Rey Noel C.
Dotosme, James Paul B.
Lauzon, Chester Josh C.
Perito, Isiah Thomas A.
Talingting, Ken Ira L.

Instructor: Mrs. Susan S. Caluya

Submission Date: October 2, 2026

Table of Contents

1 Introduction.....	1
1.1 Background.....	1
1.2 Problem Statement.....	1
1.3 Overview of the Solution.....	1
2 Project Objectives.....	2
2.1 General Objective.....	2
2.2 Specific Objectives.....	2
3 Scope and Limitations.....	3
3.1 Scope.....	3
3.2 Limitations.....	3
4 Formal Language Definition.....	4
4.1 Organization-Defined ID Structure.....	4
4.2 Definition of the Language L	4
4.3 Properties of L	4
5 Alphabet and Strings.....	5
5.1 The Alphabet Σ	5
5.2 Strings over Σ	5
6 Accepted and Rejected Input Examples.....	5
6.1 Accepted Strings.....	6
6.2 Rejected Strings.....	6
7 Regular Expression.....	7
7.1 The Regular Expression.....	7
7.2 Components of the Expression.....	7
7.3 Representation in the Program and the Test Oracle.....	7
8 NFA Formal Definition.....	7
8.1 Construction from the Regular Expression.....	7
8.2 Formal Definition.....	8
8.3 ϵ -Closure and Move.....	8
9 NFA Transition Table and Diagram.....	9
9.1 Transition Table.....	9
9.2 State Diagram.....	9
10 NFA-to-DFA Conversion.....	10
10.1 Subset Construction Algorithm.....	10
10.2 Worked Steps.....	10
10.3 Conversion Trace.....	11
10.4 Result of the Conversion.....	11
11 DFA Transition Table and Diagram.....	12
11.1 Formal Definition.....	12
11.2 Transition Table.....	12
11.3 State Diagram.....	12
12 DFA Minimization.....	13
12.1 Method.....	13
12.2 Step 1 — Unreachable States.....	13
12.3 Step 2 — Initial Partition.....	13
12.4 Step 3 — Refinement of Partitions.....	13
12.5 Steps 4 and 5 — Equivalent States and State Merging.....	14

12.6	Equivalence of the DFA and the Minimized DFA.....	15
13	Minimized DFA Diagram.....	15
13.1	Formal Definition.....	15
13.2	Transition Table.....	15
13.3	State Diagram.....	16
13.4	Meaning of the States.....	16
14	System Design.....	16
14.1	Layered Architecture.....	16
14.2	Module Responsibilities.....	17
14.3	Validation Flow.....	17
14.4	One Run Engine.....	18
14.5	User Interface Design.....	18
14.6	Tools and Technologies.....	19
15	Implementation / Source Code.....	19
15.1	Language Definition — the Single Source of Truth.....	19
15.2	Layer 1 — Alphabet Check.....	20
15.3	ϵ -Closure, Move and Thompson's Construction.....	20
15.4	Subset Construction.....	21
15.5	Minimization.....	21
15.6	Equivalence Check.....	22
15.7	Run Engine.....	22
15.8	Running the Application and the Tests.....	23
16	Test Cases and Results.....	23
16.1	Testing Strategy.....	23
16.2	Results of the Predefined Test Cases.....	24
16.3	Sample Transition Traces.....	24
16.4	Verification of Consistency between the Representations.....	25
17	Screenshots of the Working System.....	25
17.1	Simulate View.....	26
17.2	Theory View.....	30
17.3	Tests View.....	34
18	Discussion of Results.....	35
18.1	Correctness of the Recognizer.....	35
18.2	Distribution of Rejection Outcomes.....	35
18.3	Consistency between the Formal Material and the Implementation.....	36
18.4	Observations on the System.....	36
19	Conclusion.....	36
20	References.....	37
21	Individual Contribution Matrix.....	38

1 Introduction

1.1 Background

Many organizations identify their employees through structured identification codes. When such codes are typed by hand, for example by human-resources or administrative staff, errors such as a missing separator, a lowercase prefix, a transposed digit group or an extra character are common, and a malformed ID that enters a record system is difficult to trace later. Checking that a string has the agreed structure is therefore a small but real recognition problem.

Automata theory provides an exact way to state and solve this problem. The set of valid IDs is a formal language over a finite alphabet; because the format is fixed, the language is regular and can be described by a regular expression and recognized by a finite automaton. The same language can be represented in several equivalent forms — a regular expression, a nondeterministic finite automaton (NFA), a deterministic finite automaton (DFA) and a minimal DFA — and the course requires that the conversion between these forms be shown, not merely the final recognizer.

1.2 Problem Statement

The organization in this project issues employee IDs of the form EMP-YYYY-NNNN: the fixed uppercase prefix EMP, a hyphen, a four-digit year field, a second hyphen and a four-digit employee number. The task is to build a system that decides, for any string a user types, whether it is a well-formed employee ID, and that shows *how* the decision is reached — the state reached after each symbol, the transition taken and, when the input is rejected, the exact position and reason.

1.3 Overview of the Solution

The project, *Employee ID Validator: Smart Pattern Recognition System*, is a desktop application written in Python with a PySide6 (Qt) graphical interface. Its central design decision is that the regular expression is the single source of truth: the ϵ -NFA is constructed from it by Thompson's construction, the DFA is obtained from the ϵ -NFA by subset construction, and the minimal DFA is obtained from the DFA by partition refinement. None of the automata is typed by hand, and the simulator that decides acceptance runs the derived minimal DFA. Figure 1 summarizes this pipeline; the function names are those of the source code described in Section 15.

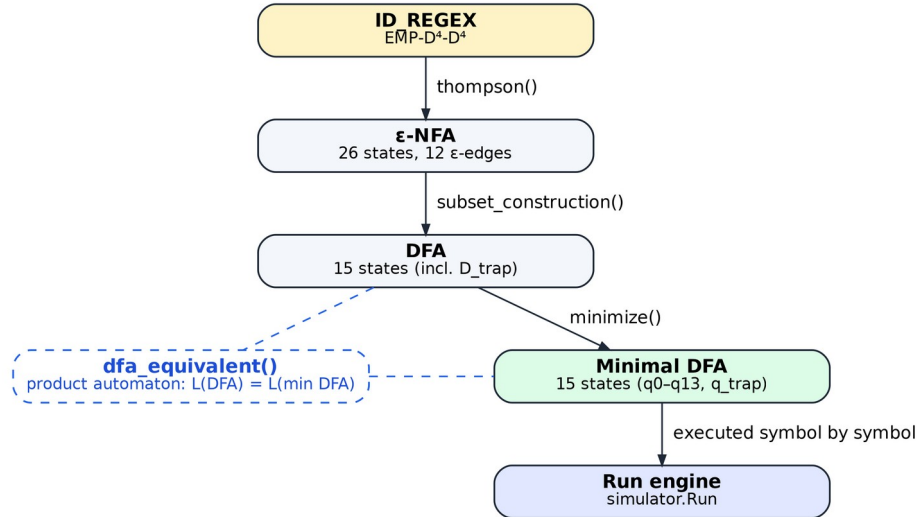


Figure 1. Derivation pipeline implemented in `app/core/pipeline.py`. Every automaton is computed from the one before it; the product-automaton check confirms that minimization preserved the language.

The application has three views. **Simulate** steps through an input symbol by symbol on the minimal DFA, the DFA before minimization or the ϵ -NFA, with an animated state diagram and a transition history. **Theory** presents every stage of the derivation — language, regular expression, ϵ -NFA, subset construction, DFA, minimization, minimal DFA and an equivalence check — generated from the same objects the simulator executes. **Tests** runs the predefined 28-case suite on all three automata and validates arbitrary batches of IDs.

The remainder of this document follows the required structure: objectives and scope (Sections 2–3), the formal treatment from the language to the minimized DFA (Sections 4–13), the system design and implementation (Sections 14–15), testing and screenshots of the working system (Sections 16–17), and the discussion, conclusion, references and contribution matrix (Sections 18–21).

2 Project Objectives

2.1 General Objective

To design and implement an employee ID validator whose recognizer is a minimized deterministic finite automaton derived, through the standard constructions of automata theory, from a formally defined regular language of organization-defined employee IDs, and to make every step of that derivation and of each recognition visible to the user.

2.2 Specific Objectives

1. Define the employee-ID format EMP-YYYY-NNNN as a formal language L over an explicitly stated alphabet Σ , with at least ten accepted and ten rejected example strings.
2. Describe L with a single regular expression and explain each of its components.
3. Construct an ϵ -NFA from the regular expression (Thompson's construction) and present its formal definition, transition table and state diagram.
4. Convert the ϵ -NFA into a DFA by subset construction, recording the move and ϵ -closure computed for every DFA state.
5. Minimize the DFA by partition refinement, documenting unreachable states, the initial partition, each refinement round, the equivalent states found and the resulting minimized DFA.

6. Implement a symbol-by-symbol simulator that uses the minimized DFA as the authoritative recognizer and reports ACCEPTED or REJECTED together with the final state and the reason.
7. Build a graphical interface that highlights the current state, shows the transition history and presents the formal material (language, RE, NFA, DFA, minimized DFA) used by the system.
8. Verify the correctness and mutual consistency of the regular expression, the three automata and the implementation through automated tests and an exact equivalence check.

3 Scope and Limitations

3.1 Scope

- **Language.** The system recognizes exactly one organization-defined format, EMP-YYYY-NNNN, over the 14-symbol alphabet $\Sigma = \{E, M, P, -, 0-9\}$.
- **Automata pipeline.** The regular expression, ϵ -NFA, DFA and minimal DFA are all computed at start-up from one definition (ID_REGEX in app/data/id_rules.py), together with the subset-construction log, the refinement rounds and a product-automaton equivalence proof.
- **Validation.** Input is checked in two layers: an alphabet check over the whole string (Layer 1) and a run of the minimal DFA (Layer 2). Five outcomes are distinguished: accepted, empty input, symbol outside Σ , no valid transition (dead state) and input ending in a non-accepting state.
- **Interface.** A PySide6 desktop application with the Simulate, Theory and Tests views, step/play/back controls, an interactive state diagram, the predefined 28-case test suite and batch validation.
- **Verification.** An automated pytest suite of 279 tests covering the core, the services, the GUI (run offscreen) and the consistency of the documentation with the code.

3.2 Limitations

- **Syntactic validation only.** The year field is *any* four digits; EMP-0000-0000 and EMP-9999-9999 are accepted. The system does not check that the year is plausible, that the ID was actually issued or that it belongs to an existing employee; there is no database or record lookup.
- **Single fixed format.** Supporting a different format requires editing ID_REGEX and the test data; the automata are then re-derived automatically, but the system does not accept several formats at once.
- **Exact, case-sensitive, ASCII-only input.** Lowercase letters, spaces, underscores and Unicode look-alike characters (for example full-width digits or a Cyrillic E) are outside Σ . Input is never trimmed, so a leading or trailing space causes rejection by design.
- **Restricted regular-expression syntax.** The expression is stored as a small abstract syntax tree with literals, a digit class, concatenation and bounded repetition. There is no text parser and no Kleene star, because the language does not need them; the Thompson construction is implemented for exactly these node types.
- **Input length.** The Simulate input field accepts at most 64 characters (setMaxLength(64)), which is ample for a 13-symbol ID; batch input is validated line by line.

- **Desktop application.** The system runs locally (Python 3.12 or later with PySide6 6.8 or later); it has no network service, user accounts or persistent storage.

4 Formal Language Definition

4.1 Organization-Defined ID Structure

An employee ID consists of five consecutive parts. Table 1 lists them with the positions they occupy; positions are numbered from 0, as they are on the input tape of the application.

Table 1. Structure of the employee ID EMP-YYYY-NNNN

Positions	Part	Content	Length	Example
0–2	Prefix	the uppercase letters E, M, P in this order	3	EMP
3	Separator	hyphen -	1	-
4–7	Year (YYYY)	any four decimal digits	4	2026
8	Separator	hyphen -	1	-
9–12	Employee number (NNNN)	any four decimal digits	4	0042

4.2 Definition of the Language L

Let Σ be the alphabet defined in Section 5 and let $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} \subset \Sigma$ denote the set of digits. The language of valid employee IDs is

$$L = \{ w \in \Sigma^* \mid w = E M P - d_1 d_2 d_3 d_4 - d_5 d_6 d_7 d_8, \text{ where } d_i \in D \text{ for } i = 1, \dots, 8 \}$$

Equivalently, $L = \{EMP\} \cdot \{-\} \cdot D^4 \cdot \{-\} \cdot D^4$, where $\cdot$ denotes concatenation of languages and D^4 is the set of all strings of exactly four digits.

4.3 Properties of L

- **Fixed length.** Every word of L has length $|w| = 3 + 1 + 4 + 1 + 4 = 13$.
- **Finite size.** The eight digit positions are independent, so $|L| = 10^8 = 100,000,000$ words.
- **Regularity.** Every finite language is regular; L is in addition described directly by the regular expression of Section 7, so it is recognized by a finite automaton.
- **Membership is positional.** Whether a symbol is acceptable depends only on how many symbols have been read correctly so far. A recognizer therefore needs no stack or unbounded counter: fourteen counting states (0 to 13 symbols matched) plus one dead state suffice, which is exactly the minimal DFA obtained in Section 13.

The empty string ϵ is not in L, and no proper prefix or extension of a word in L is in L. These two facts explain the rejection outcomes of the simulator: an input that stops early ends in a non-accepting state, while an input with a wrong or extra symbol falls into the dead state.

5 Alphabet and Strings

5.1 The Alphabet Σ

The alphabet is the set of all symbols that may appear in a valid ID. It is computed from the regular expression (`alphabet_of(ID_REGEX)`), so it cannot disagree with it:

$$\Sigma = \{ E, M, P, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}, \quad |\Sigma| = 14$$

Table 2. Symbols of Σ by role

Group	Symbols	Count	Used at positions
Prefix letters	E, M, P	3	0, 1, 2
Separator	- (hyphen-minus, U+002D)	1	3, 8
Digits	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	10	4–7, 9–12

5.2 Strings over Σ

A string over Σ is a finite sequence of symbols of Σ ; Σ^* is the set of all such strings, including the empty string ϵ . The length $|w|$ of a string is its number of symbols. L is a subset of Σ^* , so three kinds of input must be distinguished, as Table 3 illustrates.

Table 3. Classification of input strings

Kind of input	Example	In Σ^* ?	In L ?	Handled by
Valid ID	EMP-2026-0042	yes	yes	Layer 2: run ends in the accepting state
String over Σ with the wrong structure	EMP2026-0001	yes	no	Layer 2: run reaches the dead state
Prefix of a valid ID	EMP-2026-1	yes	no	Layer 2: run ends in a non-accepting state
Empty string ϵ	(empty)	yes	no	rejected before the run
Contains a symbol outside Σ	emp-2026-0001	no	no	Layer 1: alphabet check

A string that contains a symbol outside Σ is not a word over Σ at all, so the transition function δ of the automaton is not defined on it. The implementation therefore checks the *whole* input against Σ first and lists every offending symbol with its position, before the automaton is started. Lowercase letters, spaces, underscores and visually similar Unicode characters (full-width digits, Arabic-Indic digits, look-alike dashes, Cyrillic homographs) are all outside Σ and are rejected at this layer.

6 Accepted and Rejected Input Examples

The examples below are the predefined test suite of the project (`app/data/test_cases.py`): 10 strings that must be accepted and 18 that must be rejected, covering every part of the format. The verdicts actually produced by the system are reported in Section 16.

6.1 Accepted Strings

Table 4. Accepted strings (10)

No.	Test ID	String	Note
1	TC-VAL-01	EMP-2026-0001	Standard valid ID
2	TC-VAL-02	EMP-2026-0042	Standard valid ID
3	TC-VAL-03	EMP-2026-9999	Maximum sequence number
4	TC-VAL-04	EMP-2000-0000	Boundary year 2000
5	TC-VAL-05	EMP-2099-1234	Boundary year 2099
6	TC-VAL-06	EMP-1999-5555	Historical year 1999
7	TC-VAL-07	EMP-2024-8765	Recent year 2024
8	TC-VAL-08	EMP-2025-0101	Valid sequence
9	TC-VAL-09	EMP-2027-3333	Future year 2027
10	TC-VAL-10	EMP-2030-7777	Future year 2030

6.2 Rejected Strings

Table 5. Rejected strings (18)

No.	Test ID	String	Category	Reason for rejection
1	TC-INV-01	AXP-2026-0001	Invalid Prefix	Prefix is not 'EMP'
2	TC-INV-02	emp-2026-0001	Invalid Prefix	Prefix must be uppercase
3	TC-INV-03	EM-2026-0001	Invalid Prefix	Prefix too short
4	TC-INV-04	EMPP-2026-0001	Invalid Prefix	Prefix too long
5	TC-INV-05	EMP2026-0001	Missing Separator	Missing first hyphen
6	TC-INV-06	EMP-20260001	Missing Separator	Missing second hyphen
7	TC-INV-07	EMP_2026_0001	Wrong Separator	Underscore instead of hyphen
8	TC-INV-08	EMP- -2026-0001	Extra Separator	Consecutive hyphens
9	TC-INV-09	EMP-26-0001	Invalid Year	Year too short (2 digits)
10	TC-INV-10	EMP-20261-0001	Invalid Year	Year too long (5 digits)
11	TC-INV-11	EMP-202A-0001	Invalid Year	Non-numeric character in year
12	TC-INV-12	EMP-2026-1	Invalid Sequence	Sequence too short
13	TC-INV-13	EMP-2026-00001	Invalid Sequence	Sequence too long
14	TC-INV-14	EMP-2026-12B4	Invalid Sequence	Letter inside numeric sequence
15	TC-INV-15	(empty)	Empty Input	Empty input string
16	TC-INV-16	'EMP-2026-0001 '	Trailing Whitespace	Trailing space character
17	TC-INV-17	' EMP-2026-0001 '	Leading Whitespace	Leading space character
18	TC-INV-18	EMP-2026-00!1	Invalid Symbol	Symbol '!' outside alphabet Sigma

Strings with leading or trailing spaces are shown in quotation marks so that the space is visible; (empty) denotes the empty string ϵ .

7 Regular Expression

7.1 The Regular Expression

Using only the operators of the course — union ($\cup$), concatenation ($\cdot$) and, as an abbreviation, bounded repetition written as a superscript — the language L is described by

$$\text{EMP-D}^4\text{-D}^4 = \text{E} \cdot \text{M} \cdot \text{P} \cdot \text{--} \cdot \text{D} \cdot \text{D} \cdot \text{D} \cdot \text{D} \cdot \text{--} \cdot \text{D} \cdot \text{D} \cdot \text{D} \cdot \text{D}$$

where $D = (0 \cup 1 \cup 2 \cup 3 \cup 4 \cup 5 \cup 6 \cup 7 \cup 8 \cup 9)$

The superscript denotes that many copies concatenated ($D^4 = D \cdot D \cdot D \cdot D$), so the expression is an abbreviation of a plain concatenation of thirteen atoms. The digit atom D is the union of the ten digit symbols. Since the expression contains no Kleene star, every word it denotes has the same length, 13, in agreement with Section 4.3.

7.2 Components of the Expression

Table 6. Components of the regular expression

Positions	Component	Meaning	Matches	Does not match
0–2	E·M·P	fixed uppercase prefix	EMP	emp, EM, AXP, EMPP
3	-	first separator	-	_, missing hyphen
4–7	D ⁴	exactly four digits (year YYYY)	2026	26, 20261, 202A
8	-	second separator	-	--, missing hyphen
9–12	D ⁴	exactly four digits (number NNNN)	0042	1, 00001, 12B4

In the application these five parts are drawn as coloured bars under the input tape, so the user can see which component of the expression each symbol is being matched against (Figure 9 and Figure 16).

7.3 Representation in the Program and the Test Oracle

In the source code the expression is not stored as text but as a small abstract syntax tree built from four node types — `Lit` (a single symbol), `Class` (a union of symbols, here D), `Concat` and `Repeat` (Listing 1 in Section 15). The alphabet, the length 13 and the coloured segments are computed from this tree.

The same language can be written in Python's notation as `EMP-[0-9]{4}-[0-9]{4}`. This pattern uses shorthand that is not part of the course notation, and it is used **only as a test oracle**, always with `re.fullmatch` (an anchored pattern `^...$` would wrongly accept a string with a trailing newline). Acceptance in the application is never decided by Python's regular-expression engine; it is decided by the minimal DFA.

8 NFA Formal Definition

8.1 Construction from the Regular Expression

The NFA is produced from the regular expression by Thompson's construction (`thompson()` in `app/core/nfa.py`, Listing 4). Each atom of the expression — a literal symbol, or the digit class D — becomes a two-state fragment joined by its symbol(s), and consecutive fragments are joined by an ϵ -transition from the final state of one to the start state of the next. The digit class is Thompson's union ($0 \cup 1 \cup \dots \cup 9$) collapsed into a single fragment with ten parallel edges, the usual character-class shortcut; expanding it into ten separate union branches would add about 170 states without changing the language.

The thirteen atoms of the expression therefore yield $13 \times 2 = 26$ states and $13 - 1 = 12$ ϵ -transitions. Table 7 shows the fragment created for each atom.

Table 7. Thompson fragments of the thirteen atoms

Atom	Symbols	Fragment	ϵ -link to next fragment	Atom	Symbols	Fragment	ϵ -link to next fragment
1	E	$n0 \rightarrow n1$	$n1 -\epsilon \rightarrow n2$	8	0-9	$n14 \rightarrow n15$	$n15 -\epsilon \rightarrow n16$
2	M	$n2 \rightarrow n3$	$n3 -\epsilon \rightarrow n4$	9	-	$n16 \rightarrow n17$	$n17 -\epsilon \rightarrow n18$
3	P	$n4 \rightarrow n5$	$n5 -\epsilon \rightarrow n6$	10	0-9	$n18 \rightarrow n19$	$n19 -\epsilon \rightarrow n20$
4	-	$n6 \rightarrow n7$	$n7 -\epsilon \rightarrow n8$	11	0-9	$n20 \rightarrow n21$	$n21 -\epsilon \rightarrow n22$
5	0-9	$n8 \rightarrow n9$	$n9 -\epsilon \rightarrow n10$	12	0-9	$n22 \rightarrow n23$	$n23 -\epsilon \rightarrow n24$
6	0-9	$n10 \rightarrow n11$	$n11 -\epsilon \rightarrow n12$	13	0-9	$n24 \rightarrow n25$	— (n25 is accepting)
7	0-9	$n12 \rightarrow n13$	$n13 -\epsilon \rightarrow n14$				

8.2 Formal Definition

The resulting ϵ -NFA is the five-tuple

$$\begin{aligned}
 M_{\text{NFA}} &= (Q, \Sigma, \delta, q_0, F) \\
 Q &= \{n0, n1, \dots, n25\} \quad (26 \text{ states}) \\
 \Sigma &= \{E, M, P, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} \quad (14 \text{ symbols}) \\
 q_0 &= n0 \\
 F &= \{n25\}
 \end{aligned}$$

with the transition function $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$, where $P(Q)$ is the power set of Q , given in full in Table 8. For a state with an outgoing symbol edge, δ maps the symbol(s) on that edge to a one-element set and every other symbol to $\emptyset$; for a state with an ϵ -edge, $\delta(q, \epsilon)$ is the one-element set of the next fragment's start state. Unlike a DFA, an NFA has no dead state: a missing move simply means the empty set, so a run that has no move is abandoned.

8.3 ϵ -Closure and Move

Two operations are used both to simulate the NFA and to convert it to a DFA:

- **ϵ -closure(S)** — the set of states reachable from the states of S using ϵ -transitions only (including S itself);
- **move(S, a)** — the set of states reachable from a state of S by one transition on the symbol a .

The NFA reads a symbol a from a set of current states S by computing ϵ -closure(move(S, a)), and it accepts an input if, after the last symbol, the current set contains the accepting state $n25$. Because the fragments are glued by ϵ -edges, the closure is genuinely exercised: after reading E from $\{n0\}$, move gives $\{n1\}$, and the ϵ -closure adds $n2$ to give $\{n1, n2\}$. The NFA can be run directly in the application by choosing the model ϵ -NFA in the Simulate view (Figure 14).

9 NFA Transition Table and Diagram

9.1 Transition Table

Table 8 lists δ for all 26 states. The arrow marks the start state and the asterisk the accepting state; each cell is a set of states, and the column 0–9 stands for ten identical columns, one per digit. The last column is the ϵ -move.

Table 8. Transition table of the ϵ -NFA (generated by `grouped_table(nfa)`)

State	E	M	P	-	0–9	ϵ
$\rightarrow$ n0	{n1}	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
n1	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n2}
n2	$\emptyset$	{n3}	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
n3	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n4}
n4	$\emptyset$	$\emptyset$	{n5}	$\emptyset$	$\emptyset$	$\emptyset$
n5	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n6}
n6	$\emptyset$	$\emptyset$	$\emptyset$	{n7}	$\emptyset$	$\emptyset$
n7	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n8}
n8	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n9}	$\emptyset$
n9	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n10}
n10	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n11}	$\emptyset$
n11	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n12}
n12	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n13}	$\emptyset$
n13	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n14}
n14	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n15}	$\emptyset$
n15	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n16}
n16	$\emptyset$	$\emptyset$	$\emptyset$	{n17}	$\emptyset$	$\emptyset$
n17	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n18}
n18	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n19}	$\emptyset$
n19	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n20}
n20	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n21}	$\emptyset$
n21	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n22}
n22	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n23}	$\emptyset$
n23	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n24}
n24	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n25}	$\emptyset$
n25 *	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

9.2 State Diagram

Figure 2 is drawn from the same NFA object. Solid edges are symbol transitions, dashed blue edges are ϵ -transitions; the diagram reads left to right on the first row and continues in a serpentine path, ending in the accepting state n25 (double circle).

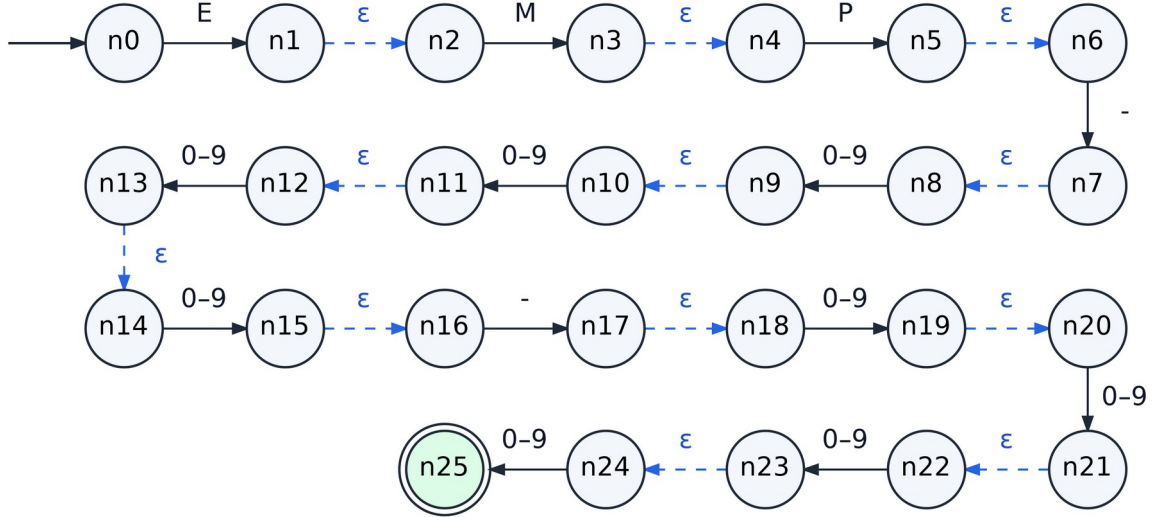


Figure 2. State diagram of the ϵ -NFA (26 states, 12 ϵ -transitions) built from $EMP-D^4-D^4$ by Thompson's construction.

10 NFA-to-DFA Conversion

10.1 Subset Construction Algorithm

The ϵ -NFA is converted to a DFA by the Rabin–Scott subset construction (`subset_construction()` in `app/core/nfa.py`, Listing 5). Each DFA state stands for a set of NFA states:

1. The start state of the DFA is $D0 = \epsilon\text{-closure}(\{n0\})$.
2. For every unprocessed DFA state S and every symbol $a \in \Sigma$, compute $\text{move}(S, a)$ and then its ϵ -closure.
3. Every new non-empty set obtained becomes a new DFA state, named $D1, D2, \dots$ in the order of discovery; the empty set becomes one explicit dead state, D_{trap} , whose transitions all lead back to itself.
4. A DFA state is accepting if and only if its set contains an accepting NFA state ($n25$).
5. The construction ends when no unprocessed set remains. The resulting transition function is total: every pair (state, symbol) has exactly one successor.

10.2 Worked Steps

The first steps of the conversion, followed by hand, are:

$$\begin{aligned}
 D0 &= \epsilon\text{-closure}(\{n0\}) = \{n0\} \\
 \text{move}(D0, E) &= \{n1\}, \quad \epsilon\text{-closure}(\{n1\}) = \{n1, n2\} = D1 \\
 \text{move}(D0, a) &= \emptyset \text{ for every } a \neq E \Rightarrow \delta(D0, a) = D_{\text{trap}} \\
 \text{move}(D1, M) &= \{n3\}, \quad \epsilon\text{-closure}(\{n3\}) = \{n3, n4\} = D2 \\
 \text{move}(D2, P) &= \{n5\}, \quad \epsilon\text{-closure}(\{n5\}) = \{n5, n6\} = D3 \\
 \text{move}(D3, -) &= \{n7\}, \quad \epsilon\text{-closure}(\{n7\}) = \{n7, n8\} = D4 \\
 \text{move}(D4, d) &= \{n9\} \text{ for every digit } d, \quad \epsilon\text{-closure}(\{n9\}) = \{n9, n10\} = D5
 \end{aligned}$$

Continuing in the same way produces D6 to D13; D13 = {n25} contains the accepting NFA state and is the only accepting DFA state. The complete trace, as recorded by the program, is given in Table 9. Symbols that lead to the same set are grouped in one row.

10.3 Conversion Trace

Table 9. Subset construction trace (recorded by `subset_construction()`; * marks the accepting DFA state)

DFA state	NFA subset	Symbols a	move(S, a)	ϵ -closure	Target
D0	{n0}	M, P, -, 0–9	$\emptyset$	$\emptyset$	D_trap
		E	{n1}	{n1, n2}	D1
D1	{n1, n2}	E, P, -, 0–9	$\emptyset$	$\emptyset$	D_trap
		M	{n3}	{n3, n4}	D2
D2	{n3, n4}	E, M, -, 0–9	$\emptyset$	$\emptyset$	D_trap
		P	{n5}	{n5, n6}	D3
D3	{n5, n6}	-	{n7}	{n7, n8}	D4
		E, M, P, 0–9	$\emptyset$	$\emptyset$	D_trap
D4	{n7, n8}	E, M, P, -	$\emptyset$	$\emptyset$	D_trap
		0–9	{n9}	{n9, n10}	D5
D5	{n9, n10}	E, M, P, -	$\emptyset$	$\emptyset$	D_trap
		0–9	{n11}	{n11, n12}	D6
D6	{n11, n12}	E, M, P, -	$\emptyset$	$\emptyset$	D_trap
		0–9	{n13}	{n13, n14}	D7
D7	{n13, n14}	E, M, P, -	$\emptyset$	$\emptyset$	D_trap
		0–9	{n15}	{n15, n16}	D8
D8	{n15, n16}	-	{n17}	{n17, n18}	D9
		E, M, P, 0–9	$\emptyset$	$\emptyset$	D_trap
D9	{n17, n18}	E, M, P, -	$\emptyset$	$\emptyset$	D_trap
		0–9	{n19}	{n19, n20}	D10
D10	{n19, n20}	E, M, P, -	$\emptyset$	$\emptyset$	D_trap
		0–9	{n21}	{n21, n22}	D11
D11	{n21, n22}	E, M, P, -	$\emptyset$	$\emptyset$	D_trap
		0–9	{n23}	{n23, n24}	D12
D12	{n23, n24}	E, M, P, -	$\emptyset$	$\emptyset$	D_trap
		0–9	{n25}	{n25}	D13
D13 *	{n25}	E, M, P, -, 0–9	$\emptyset$	$\emptyset$	D_trap
D_trap	$\emptyset$	E, M, P, -, 0–9	$\emptyset$	$\emptyset$	D_trap

10.4 Result of the Conversion

The construction discovers 14 non-empty subsets (D0–D13) and the empty subset, giving a complete DFA with 15 states. Every subset contains at most two NFA states — a fragment's end state and the next fragment's start state joined by ϵ — because the NFA built from this expression is a single chain: for each subset and symbol there is at most one symbol edge to follow. The NFA's nondeterminism here lies entirely in its ϵ -transitions, and the subset construction removes them. (The test suite also

converts a genuinely nondeterministic NFA, in which one symbol leads to several states, to confirm that the implementation handles the general case.)

11 DFA Transition Table and Diagram

11.1 Formal Definition

$M_{DFA} = (Q, \Sigma, \delta, q_0, F)$
 $Q = \{D0, D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D11, D12, D13, D_trap\}$ (15 states)
 $\Sigma = \{E, M, P, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (14 symbols)
 $q_0 = D0$
 $F = \{D13\}$

The transition function $\delta : Q \times \Sigma \rightarrow Q$ is total, as the DFA class in `app/core/dfa.py` enforces when the automaton is created: every undefined move of the NFA becomes an explicit transition to `D_trap`.

11.2 Transition Table

Table 10. Transition table of the DFA obtained by subset construction

State	E	M	P	-	0–9
→ D0	D1	D_trap	D_trap	D_trap	D_trap
D1	D_trap	D2	D_trap	D_trap	D_trap
D2	D_trap	D_trap	D3	D_trap	D_trap
D3	D_trap	D_trap	D_trap	D4	D_trap
D4	D_trap	D_trap	D_trap	D_trap	D5
D5	D_trap	D_trap	D_trap	D_trap	D6
D6	D_trap	D_trap	D_trap	D_trap	D7
D7	D_trap	D_trap	D_trap	D_trap	D8
D8	D_trap	D_trap	D_trap	D9	D_trap
D9	D_trap	D_trap	D_trap	D_trap	D10
D10	D_trap	D_trap	D_trap	D_trap	D11
D11	D_trap	D_trap	D_trap	D_trap	D12
D12	D_trap	D_trap	D_trap	D_trap	D13
D13 *	D_trap	D_trap	D_trap	D_trap	D_trap
D_trap	D_trap	D_trap	D_trap	D_trap	D_trap

11.3 State Diagram

In Figure 3 each state is labelled with its name and, below it, the set of NFA states it represents. The dashed grey arrows are the transitions to `D_trap`: from each live state, every symbol other than the one on its solid outgoing edge leads to `D_trap`, and `D_trap` loops to itself on every symbol of Σ .

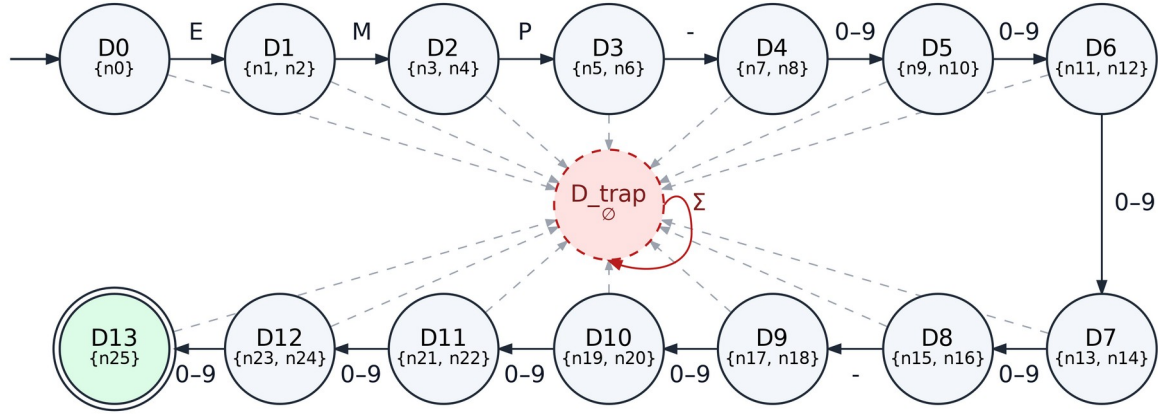


Figure 3. State diagram of the DFA before minimization (15 states). Dashed grey edges: all remaining symbols of Σ lead to D_trap .

12 DFA Minimization

12.1 Method

The DFA is minimized with Moore's partition-refinement algorithm (`minimize()` in `app/core/minimizer.py`, Listing 6). Two states are equivalent if no input string leads one of them to acceptance and the other to rejection; the algorithm computes this relation by successive refinement:

1. Remove every state that is not reachable from the start state.
2. Start from the partition $P_0 = \{ F, Q \setminus F \}$ of accepting and non-accepting states.
3. Refine: two states stay in the same block only if, for every symbol $a \in \Sigma$, their successors lie in the same block of the current partition. Each pass that splits at least one block is recorded as a round $P_1, P_2, \dots$
4. Stop when a pass splits nothing. Each block of the final partition is one state of the minimized DFA; equivalent states (blocks with more than one member) are merged.
5. Rename the blocks in breadth-first order from the start state as $q_0, q_1, \dots$, and name the dead block q_trap .

Moore's algorithm runs in $O(n^2 \cdot |\Sigma|)$ time; Hopcroft's worklist variant computes the same partition in $O(n \log n)$, but with $n = 15$ states the difference is immaterial, so the simpler algorithm was chosen.

12.2 Step 1 — Unreachable States

A breadth-first search from D_0 reaches all 15 states, so the set of unreachable states is **empty**. This is expected, since subset construction only creates subsets that are reached from the start state.

12.3 Step 2 — Initial Partition

$$P_0 = \{ F, Q \setminus F \} = \{ \{D_{13}\}, \{D_0, D_1, \dots, D_{12}, D_trap\} \}$$

12.4 Step 3 — Refinement of Partitions

Table 11 gives every partition computed by the program. In round 1 the block of non-accepting states splits because D_{12} is the only one of its members that reaches the accepting block, on any digit. Each following round isolates exactly one more state, working backwards along the chain: the state that

reaches the newly isolated state in one step is separated from the rest. In round 13 the last two members of the large block, D0 and D_trap, are separated because on E the state D0 reaches D1, which is already in a block of its own, while D_trap stays in D_trap.

Table 11. Partition refinement rounds (computed by `minimize()`)

Round	Blocks	Partition
P0	2	{D0–D12, D_trap} {D13}
P1	3	{D0–D11, D_trap} {D12} {D13}
P2	4	{D0–D10, D_trap} {D11} {D12} {D13}
P3	5	{D0–D9, D_trap} {D10} {D11} {D12} {D13}
P4	6	{D0–D8, D_trap} {D9} {D10} {D11} {D12} {D13}
P5	7	{D0–D7, D_trap} {D8} {D9} {D10} {D11} {D12} {D13}
P6	8	{D0–D6, D_trap} {D7} {D8} {D9} {D10} {D11} {D12} {D13}
P7	9	{D0–D5, D_trap} {D6} {D7} {D8} {D9} {D10} {D11} {D12} {D13}
P8	10	{D0–D4, D_trap} {D5} {D6} {D7} {D8} {D9} {D10} {D11} {D12} {D13}
P9	11	{D0–D3, D_trap} {D4} {D5} {D6} {D7} {D8} {D9} {D10} {D11} {D12} {D13}
P10	12	{D0–D2, D_trap} {D3} {D4} {D5} {D6} {D7} {D8} {D9} {D10} {D11} {D12} {D13}
P11	13	{D0, D1, D_trap} {D2} {D3} {D4} {D5} {D6} {D7} {D8} {D9} {D10} {D11} {D12} {D13}
P12	14	{D0, D_trap} {D1} {D2} {D3} {D4} {D5} {D6} {D7} {D8} {D9} {D10} {D11} {D12} {D13}
P13	15	{D0} {D1} {D2} {D3} {D4} {D5} {D6} {D7} {D8} {D9} {D10} {D11} {D12} {D13} {D_trap}

The partition P13 consists of 15 single-state blocks, and a further pass splits nothing, so the refinement has stabilized after 13 rounds.

12.5 Steps 4 and 5 — Equivalent States and State Merging

No block of the final partition has more than one member: **there are no equivalent states, and no states are merged.** The DFA produced by subset construction is already minimal ($15 \rightarrow 15$ states). This is a result of the algorithm, not a skipped step, and it has a direct explanation. From each live state the shortest accepted continuation has a different length — from D_i exactly $13 - i$ more symbols are needed — so any two live states are distinguished by a suitable string of digits and separators, and D_trap, from which nothing can be accepted, differs from all of them. Table 12 lists these lengths, computed from the automaton, together with the new name of each state.

Table 12. State correspondence and distinguishing measure

DFA state	Minimized state	Fewest symbols to acceptance	DFA state	Minimized state	Fewest symbols to acceptance
D0	q0	13	D8	q8	5
D1	q1	12	D9	q9	4
D2	q2	11	D10	q10	3
D3	q3	10	D11	q11	2
D4	q4	9	D12	q12	1
D5	q5	8	D13	q13	0
D6	q6	7	D_trap	q_trap	∞ (dead)
D7	q7	6			

Renaming is therefore the only change: D_i becomes q_i for $i = 0, \dots, 13$, and D_{trap} becomes q_{trap} . Because merging is never exercised by this language, the project's test suite separately minimizes a deliberately redundant DFA, containing two pairs of equivalent states and an unreachable state, to confirm that removal and merging are implemented correctly.

12.6 Equivalence of the DFA and the Minimized DFA

That minimization preserved the language is checked exactly, not by sampling, with the product-automaton method (`dfa_equivalent()`, Listing 7). A breadth-first search explores the pairs (p, q) reachable when both automata read the same input; the languages differ if and only if some reachable pair has exactly one accepting component, and the search then returns the shortest distinguishing word. For the subset DFA and the minimized DFA the search explores 15 pairs and finds none, so the two automata are **equivalent**.

13 Minimized DFA Diagram

13.1 Formal Definition

$M_{\text{min}} = (Q, \Sigma, \delta, q_0, F)$
 $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}, q_{12}, q_{13}, q_{\text{trap}}\}$ (15 states)
 $\Sigma = \{E, M, P, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (14 symbols)
 $q_0 = q_0$
 $F = \{q_{13}\}$

13.2 Transition Table

Table 13 is the transition function executed by the simulator. The rows q_0 to q_{13} count how many symbols of a valid ID have been matched; every undefined move leads to the dead state q_{trap} .

Table 13. Transition table of the minimized DFA (the authoritative recognizer)

State	E	M	P	-	0-9
$\rightarrow q_0$	q_1	q_{trap}	q_{trap}	q_{trap}	q_{trap}
q_1	q_{trap}	q_2	q_{trap}	q_{trap}	q_{trap}
q_2	q_{trap}	q_{trap}	q_3	q_{trap}	q_{trap}
q_3	q_{trap}	q_{trap}	q_{trap}	q_4	q_{trap}
q_4	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_5
q_5	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_6
q_6	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_7
q_7	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_8
q_8	q_{trap}	q_{trap}	q_{trap}	q_9	q_{trap}
q_9	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_{10}
q_{10}	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_{11}
q_{11}	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_{12}
q_{12}	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_{13}
q_{13}^*	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_{trap}
q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_{trap}

13.3 State Diagram

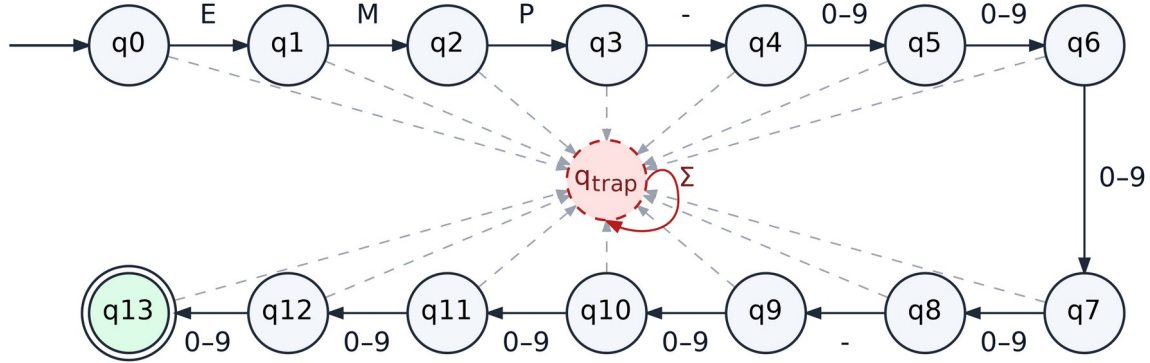


Figure 4. State diagram of the minimized DFA (15 states). q0 is the start state, q13 the only accepting state; dashed grey edges are the transitions to q_trap.

13.4 Meaning of the States

Table 14. Interpretation of the states of the minimized DFA (d denotes any digit)

State	Input read so far	Next symbol expected
q0	nothing read yet	E
q1	E	M
q2	EM	P
q3	EMP	-
q4	EMP-	a digit (1st year digit)
q5	EMP-d	a digit
q6	EMP-dd	a digit
q7	EMP-ddd	a digit
q8	EMP-dddd	-
q9	EMP-dddd-	a digit (1st number digit)
q10	EMP-dddd-d	a digit
q11	EMP-dddd-dd	a digit
q12	EMP-dddd-ddd	a digit
q13	a complete ID (accepting)	end of input
q_trap	an invalid prefix (dead state)	— (no input can lead to acceptance)

This reading of the states is what the application uses to explain a rejection. For example, when the input EMP2026-0001 is run, the automaton is in q3 after EMP, expects - and reads 2, so it moves to q_trap and the message reads “q3 expects '-' but read '2'” (Figure 10).

14 System Design

14.1 Layered Architecture

The system is organized in four layers (Figure 5). Dependencies point downwards only: the GUI calls the services, the services call the automata core, and the core reads the language definition from the data layer. The core does not import PySide6, so every automaton and the simulator can be run and

tested from the command line, and the GUI contains no automata algorithm — it only displays the structured results it receives.

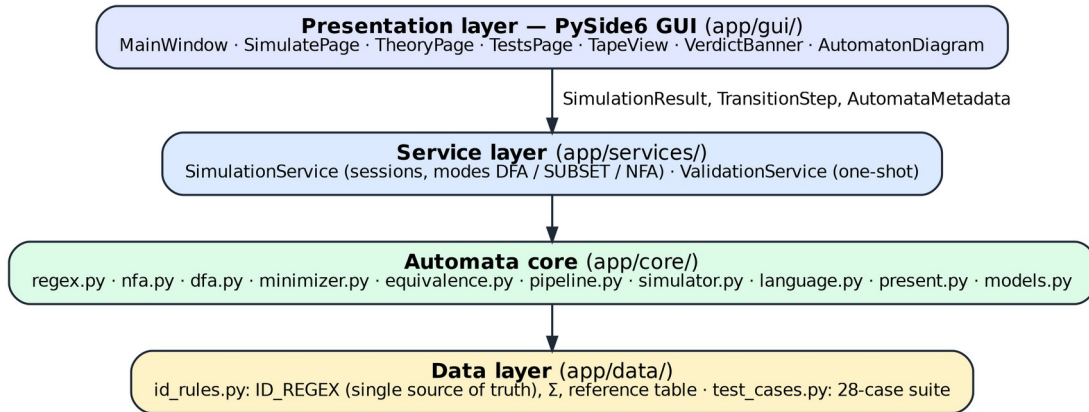


Figure 5. Layered architecture of the Employee ID Validator.

14.2 Module Responsibilities

Table 15. Modules and their responsibilities

Module	Responsibility
app/data/id_rules.py	ID_REGEX (the single source of truth), the derived alphabet Σ and length, the named RE segments, and a hand-written reference DFA used only for cross-checking
app/data/test_cases.py	The 28 predefined test cases with expected verdicts and reasons
app/core/regex.py	RE abstract syntax tree (Lit, Class, Concat, Repeat); alphabet, length and formal notation
app/core/nfa.py	ϵ -NFA class, ϵ -closure, move, Thompson construction, subset construction with step log
app/core/dfa.py	Validated DFA with a total transition function; dead-state detection; expected symbols
app/core/minimizer.py	Moore partition refinement with recorded rounds, merging and renaming
app/core/equivalence.py	Exact DFA equivalence by product automaton; exhaustive agreement check
app/core/pipeline.py	Derives RE $\rightarrow$ ϵ -NFA $\rightarrow$ DFA $\rightarrow$ minimal DFA once and caches the result
app/core/language.py	Layer 1: alphabet check of the whole input; wording of expected symbols
app/core/simulator.py	Run engine: symbol-by-symbol execution, trace and result for a DFA or an NFA
app/core/models.py	Shared data contracts: SimulationStatus, TransitionStep, SimulationResult, AutomataMetadata
app/core/present.py	Tables, diagram graphs and the Markdown report derived from the core objects
app/services/	SimulationService (step-by-step sessions in three modes) and ValidationService (one-shot)
app/gui/	Main window, Simulate / Theory / Tests pages, tape, verdict banner and interactive diagram widgets, theme

14.3 Validation Flow

Each input passes through three layers. Layer 1 checks that the input is non-empty and that every symbol belongs to Σ ; Layer 2 runs the minimized DFA one symbol at a time and stops as soon as the dead state is entered; Layer 3 presents the outcome. Figure 6 shows the decision procedure, and Table 16 lists the five possible outcomes defined in app/core/models.py.

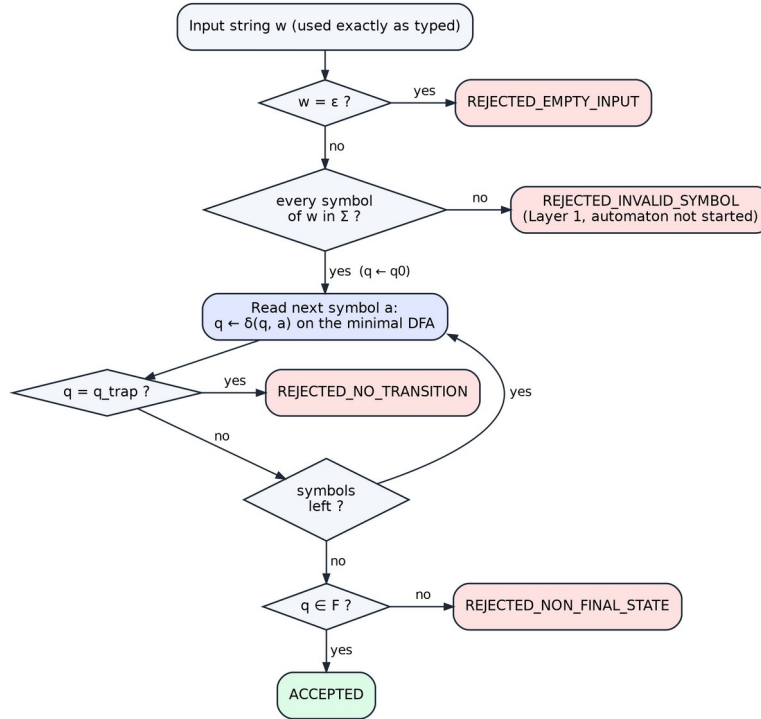


Figure 6. Validation flow of the run engine (app/core/simulator.py).

Table 16. Simulation outcomes (SimulationStatus)

Status	Condition	Automaton started?
ACCEPTED	all symbols consumed and the final state is in F (q13)	yes
REJECTED_EMPTY_INPUT	the input is the empty string	no
REJECTED_INVALID_SYMBOL	at least one symbol is not in Σ (all offenders are listed)	no
REJECTED_NO_TRANSITION	a symbol leads to the dead state q_{trap} ; the run stops there	yes
REJECTED_NON_FINAL_STATE	all symbols consumed but the final state is not accepting	yes

Every result is returned as a `SimulationResult` record containing the verdict, the status, the final state, the full transition trace (one `TransitionStep` per symbol), the position of the error, the number of symbols processed and a human-readable explanation. The GUI and the tests consume only these records.

14.4 One Run Engine

Stepping, tracing and result-building are implemented once, in the class `Run` (Listing 8). The one-shot function `simulate()` is a loop over `Run`, and every interactive session of `SimulationService` wraps a `Run`. The animated GUI and the batch validator therefore cannot disagree, and the same engine executes the minimal DFA, the DFA before minimization or the ϵ -NFA (in which case the current state is a set of NFA states).

14.5 User Interface Design

The main window has a header with the three views and remains fully keyboard-operable. Its design goals were to make the automaton visible during recognition and to present the theory from the objects actually executed:

- **Simulate** — an input field with Load, a list of example inputs and a model selector (minimal DFA, DFA before minimization, ϵ -NFA); a tape showing each symbol with its index and the RE segment it belongs to; a live read-out of the current symbol, the state change and the value of δ ; a verdict banner (READY, RUNNING, ACCEPTED, REJECTED) with the explanation; the state diagram, in which the current state is amber, visited states are tinted, the accepting state has a double ring and the dead state is dashed red; the transition history table; and Prev, Play, Step, Run all and Reset controls with a speed slider (0.5× to 4.0×). Keyboard shortcuts: Space (play/pause), → (step), ← (previous step), Ctrl+R (reset).
- **Theory** — summary chips (Σ , Q , q_0 , F , δ) and eight tabs: Language, Regular expression, ϵ -NFA, Subset construction, DFA, Minimization (with a slider to replay the refinement rounds), Minimal DFA (where the path of the last run is coloured in the table) and Equivalence (with an exhaustive check that can be run on demand). A button copies the whole theory report as Markdown.
- **Tests** — the 28-case suite with expected and actual verdicts, pass/fail, whether all three automata agree, and a Simulate button for each case; and a Batch tab that validates any list of IDs, one per line, exactly as written.

Hovering over or clicking any state or transition of a diagram displays an explanation of it; the colour theme was checked for contrast by automated tests, and animations can be disabled with the environment variable CAUTOMA_REDUCE_MOTION=1.

14.6 Tools and Technologies

Table 17. Tools and technologies

Tool	Version / setting	Use in the project
Python	3.12 or later (tested with 3.13.5)	implementation language of the core, services and GUI
PySide6 (Qt for Python)	6.8 or later (tested with 6.8.2.1)	desktop graphical user interface
pytest	8 or later	automated test suite (GUI tests run with the offscreen Qt platform)
Ruff	line length 100, rules E, F, W, I, UP	linting and import ordering
Git	—	version control and team integration
Graphviz	—	rendering of the automaton figures in this document from the project's automata objects

15 Implementation / Source Code

This section presents the parts of the source code that implement the formal constructions of Sections 7–13. Every listing is copied from the repository with its original line numbers; docstrings that only repeat the surrounding text have been left in place so that the excerpts are exact.

15.1 Language Definition — the Single Source of Truth

Listing 1 defines the regular expression as an abstract syntax tree. The named parts label the coloured segments of the tape; the expression itself is their concatenation. Changing the ID format means changing only these lines: the alphabet, the automata, the tables and the diagrams are all recomputed from ID_REGEX.

Listing 1. Definition of the regular expression EMP-D⁴-D⁴ (app/data/id_rules.py, lines 17–28)

```

17 DIGITS: list[str] = [str(d) for d in range(10)] # '0' .. '9'
18
19 DIGIT = Class(tuple(DIGITS), name="D")
20 # The RE as named parts (the names only label the tape and the docs; the RE is the
  concatenation).
21 _PARTS = (
22     ("prefix", Concat((Lit("E"), Lit("M"), Lit("P")))),
23     ("separator", Lit("-")),
24     ("year YYYY", Repeat(DIGIT, 4)), # any four digits (syntactic check only, no year
  range)
25     ("separator", Lit("-")),
26     ("number NNNN", Repeat(DIGIT, 4)),
27 )
28 ID_REGEX = Concat(tuple(node for _, node in _PARTS))

```

15.2 Layer 1 — Alphabet Check

Listing 2 implements the alphabet check. It collects *every* symbol of the input that is not in Σ , with its index, so the user is told about all offending characters at once.

Listing 2. Alphabet membership and whole-string check (app/core/language.py, lines 15–23)

```

15 def is_symbol_in_alphabet(char: str) -> bool:
16     """True iff `char` is a single symbol belonging to Sigma."""
17     return len(char) == 1 and char in ALPHABET
18
19
20 def validate_symbols(input_str: str) -> tuple[bool, list[tuple[int, str]]]:
21     """(True, []) if every symbol is in Sigma, else (False, [(index, char), ...])."""
22     illegal = [(i, ch) for i, ch in enumerate(input_str) if ch not in ALPHABET]
23     return not illegal, illegal

```

15.3 ϵ -Closure, Move and Thompson's Construction

Listing 3 implements the two operations of Section 8.3. The ϵ -closure is a depth-first search over ϵ -edges; move collects the targets of one symbol from a set of states.

Listing 3. ϵ -closure and move (app/core/nfa.py, lines 80–92)

```

80 def epsilon_closure(nfa: NFA, states: Iterable[str]) -> frozenset[str]:
81     closure = set(states)
82     stack = list(closure)
83     while stack:
84         for nxt in nfa.epsilon.get(stack.pop(), ()):
85             if nxt not in closure:
86                 closure.add(nxt)
87                 stack.append(nxt)
88     return frozenset(closure)
89
90
91 def move(nfa: NFA, states: Iterable[str], symbol: str) -> frozenset[str]:
92     return frozenset(t for s in states for t in nfa.transitions[s].get(symbol, ()))

```

Listing 4 is the Thompson construction. `build()` returns the start and end state of the fragment for one node of the syntax tree: a literal or a class becomes two new states joined by its symbol(s), a repetition is expanded into a concatenation, and a concatenation joins consecutive fragments with ϵ -edges.

Listing 4. Thompson's construction from the RE syntax tree (app/core/nfa.py, lines 98–134)

```

98 def thompson(node: Node) -> NFA:
99     """
100     Build an epsilon-NFA from the RE. Every atom (Lit or Class) becomes two states joined by
101     its symbol(s); concatenation joins consecutive fragments with an epsilon edge.
102     A Class is Thompson's union ( $0 \cup 1 \cup \dots \cup 9$ ) collapsed into one atom with one edge per
103     symbol (the standard character-class shortcut, avoiding 10 parallel branches per digit).
104     """
105     counter = itertools.count()
106     trans: dict[str, dict[str, set[str]]] = {}
107     eps: dict[str, set[str]] = {}
108

```

```

109 def new() -> str:
110     s = f"n{next(counter)}"
111     trans[s], eps[s] = {}, set()
112     return s
113
114 def build(n: Node) -> tuple[str, str]:
115     match n:
116         case Lit(c):
117             s, e = new(), new()
118             trans[s][c] = {e}
119             return s, e
120         case Class(chars):
121             s, e = new(), new()
122             for c in chars:
123                 trans[s][c] = {e}
124             return s, e
125         case Repeat(inner, k):
126             return build(Concat((inner,) * k))
127         case Concat(parts):
128             frags = [build(p) for p in parts]
129             for (_, end), (start, _) in zip(frags, frags[1:]):
130                 eps[end].add(start)
131             return frags[0][0], frags[-1][1]
132
133 start, end = build(node)
134 return NFA(list(trans), trans, start, {end}, eps)

```

15.4 Subset Construction

Listing 5 is the core of the subset construction. A queue holds the subsets still to be processed; for each one, the symbols of Σ are grouped by their (move, ε -closure) outcome, which is exactly the grouping shown in Table 9. A non-empty closure seen for the first time receives the next name Dk; the empty closure is mapped to D_trap.

Listing 5. Subset construction (main loop) (app/core/nfa.py, lines 160–191)

```

160 def subset_construction(nfa: NFA) -> tuple[DFA, list[SubsetStep]]:
161     """
162     Rabin-Scott subset construction. Returns the COMPLETE DFA (the empty subset becomes an
163     explicit dead state D_trap) and a step log with move / epsilon-closure per symbol group.
164     """
165     sigma = sorted(nfa.alphabet)
166     start = nfa.start_closure()
167     names: dict[frozenset[str], str] = {start: "D0"}
168     queue = deque([start])
169     steps: list[SubsetStep] = []
170     transitions: dict[str, dict[str, str]] = {}
171     dead_needed = False
172
173     while queue:
174         current = queue.popleft()
175         groups: dict[tuple[frozenset[str], frozenset[str]], list[str]] = {}
176         for a in sigma:
177             mv = move(nfa, current, a)
178             groups.setdefault((mv, epsilon_closure(nfa, mv)), []).append(a)
179
180         moves, row = [], {}
181         for (mv, clo), symbols in groups.items():
182             if clo:
183                 if clo not in names:
184                     names[clo] = f"D{len(names)}"
185                     queue.append(clo)
186                     target = names[clo]
187             else:
188                 target, dead_needed = DEAD_NAME, True
189             moves.append(SubsetMove(tuple(symbols), mv, clo, target))
190             row.update({a: target for a in symbols})
191         transitions[names[current]] = row

```

15.5 Minimization

Listing 6 implements the refinement of Section 12. Each state is given a signature consisting of its current block and the blocks of its successors on every symbol; states with equal signatures form the blocks of the next partition. The loop stops when the number of blocks no longer grows, and every partition is stored as a round.

Listing 6. Moore partition refinement (app/core/minimizer.py, lines 49–66)

```

49 def minimize(dfa: DFA, prefix: str = "q", dead_name: str = "q_trap") -> MinimizationResult:
50     reachable = set(_reachable(dfa))
51     removed = tuple(s for s in dfa.states if s not in reachable)
52     states = [s for s in dfa.states if s in reachable]
53     sigma = sorted(dfa.alphabet)
54
55     block_of = {s: 0 if dfa.is_accepting(s) else 1 for s in states}
56     rounds = [_partition(block_of)]
57     while True:
58         ids: dict[tuple, int] = {}
59         refined = {
60             s: ids.setdefault((block_of[s], *(block_of[dfa.step(s, a)] for a in sigma)),
61                             len(ids))
62             for s in states
63         }
64         if len(ids) == len(set(block_of.values())):
65             break
66         block_of = refined
67         rounds.append(_partition(block_of))

```

15.6 Equivalence Check

Listing 7. Exact DFA equivalence by product automaton (app/core/equivalence.py, lines 23–39)

```

23 def dfa_equivalent(a: DFA, b: DFA) -> Equivalence:
24     if a.alphabet != b.alphabet:
25         raise ValueError("DFAs must share the same alphabet")
26     sigma = sorted(a.alphabet)
27     start = (a.start_state, b.start_state)
28     seen = {start}
29     queue = deque([(start, "")])
30     while queue:
31         (p, q), word = queue.popleft()
32         if a.is_accepting(p) != b.is_accepting(q):
33             return Equivalence(False, word, len(seen))
34         for s in sigma:
35             nxt = (a.step(p, s), b.step(q, s))
36             if nxt not in seen:
37                 seen.add(nxt)
38                 queue.append((nxt, word + s))
39     return Equivalence(True, None, len(seen))

```

15.7 Run Engine

Listing 8 shows how one symbol is consumed. For a DFA the next state is $\delta(\text{state}, \text{symbol})$; for the NFA it is $\epsilon\text{-closure}(\text{move}(\dots))$. Every step is recorded as a `TransitionStep`. If the new state is dead, the run finishes immediately with `REJECTED_NO_TRANSITION` and an explanation of which symbols were expected; when the last symbol has been read, `_finish_consumed()` decides between `ACCEPTED` and `REJECTED_NON_FINAL_STATE`.

Listing 8. Consuming one symbol in the run engine (app/core/simulator.py, lines 65–98)

```

65 def step(self) -> TransitionStep | None:
66     """Consume one symbol; None once the run has finished."""
67     if self.finished:
68         return None
69     m, idx = self.machine, self.cursor
70     symbol = self.input[idx]
71     src = self.state_label
72     prev = self.active
73
74     if self.is_nfa:
75         self.active = m.step(prev, symbol)
76         dst, dead = format_subset(self.active), not self.active
77         expected = m.expected(prev)
78     else:
79         nxt = m.step(src, symbol)
80         self.active = frozenset({nxt})
81         dst, dead = nxt, m.is_dead(nxt)
82         expected = m.expected(src)
83     self.cursor += 1
84
85     if dead:

```

```

86         why = f"No transition: {src} expects {describe_symbols(expected)} but read
{symbol!r}."
87     else:
88         why = f"{src} --{symbol}--> {dst}"
89         step = TransitionStep(idx + 1, symbol, src, dst, not dead, why)
90         self.trace.append(step)
91
92     if dead:
93         self._finish(SimulationStatus.REJECTED_NO_TRANSITION, dst, idx,
94                     f"No valid transition on {symbol!r} at position {idx}.",
95                     f"Rejected at position {idx}: {why}")
96     elif self.cursor == len(self.input):
97         self._finish_consumed(dst)
98     return step

```

15.8 Running the Application and the Tests

The application is started and tested from the project directory with the following commands:

```

pip install -r requirements.txt          # PySide6, pytest, ruff
python -m app.main                      # start the application
python -m pytest                        # run the automated tests
python scripts/gen_docs.py --check      # verify that the
documentation tables match the code

```

16 Test Cases and Results

16.1 Testing Strategy

Testing was carried out at every layer, from the alphabet check to the GUI. The automated suite was run on Python 3.13.5 with PySide6 6.8.2.1; all **279 tests passed** in 24.3 seconds, the linter reported no issues and the documentation-consistency check found no stale tables. Table 18 lists the test files.

Table 18. Automated test suite (279 tests, all passed)

Test file	Tests	What is verified
tests/core/test_language.py	21	Σ membership, Unicode look-alikes, whole-string alphabet check, empty input
tests/core/test_nfa_pipeline.py	14	RE notation; Thompson structure (26 states, 12 ϵ -edges); ϵ -closure; genuine nondeterminism; subset construction
tests/core/test_dfa.py	28	DFA five-tuple validation, total δ , dead state, acceptance
tests/core/test_minimization.py	7	refinement rounds, no-merge proof, real merging and unreachable removal on a redundant DFA, idempotence
tests/core/test_equivalence.py	16	product-automaton proof, reference table, 2,940 single-transition mutants, exhaustive and random comparison with the oracle
tests/core/test_simulator.py	19	Layer 1 vs Layer 2, explanations, step-by-step equals one-shot for all three models
tests/core/test_edge_cases.py	37	valid, boundary, prefix, length, separator, symbol, empty and very long inputs; independence of repeated validations
tests/core/test_models.py	3	construction and immutability of the shared data contracts
tests/services/test_services.py	21	sessions, modes, computed metadata
tests/integration/test_master_suite.py	56	the 28 predefined cases through ValidationService and SimulationService
tests/gui/test_gui.py, test_interaction.py, test_theme.py	52	views, playback, diagram highlighting, hover explanations, keyboard focus, accessible names, WCAG AA colour contrast
tests/test_docs.py	5	tables in docs/ equal the tables generated from the code

16.2 Results of the Predefined Test Cases

Table 19 shows the result of running each predefined case through the minimized DFA. All 28 cases produce the expected verdict (28/28 passed), and the ϵ -NFA, the DFA before minimization and the minimized DFA return the same verdict on every case.

Table 19. Predefined test cases: expected and actual results on the minimized DFA

Test ID	Input	Expected	Actual	Final state	Symbols read	Outcome	Result
TC-VAL-01	EMP-2026-0001	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-VAL-02	EMP-2026-0042	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-VAL-03	EMP-2026-9999	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-VAL-04	EMP-2000-0000	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-VAL-05	EMP-2099-1234	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-VAL-06	EMP-1999-5555	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-VAL-07	EMP-2024-8765	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-VAL-08	EMP-2025-0101	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-VAL-09	EMP-2027-3333	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-VAL-10	EMP-2030-7777	Accept	Accept	q13	13/13	ACCEPTED	PASS
TC-INV-01	AXP-2026-0001	Reject	Reject	—	0/13	INVALID_SYMBOL	PASS
TC-INV-02	emp-2026-0001	Reject	Reject	—	0/13	INVALID_SYMBOL	PASS
TC-INV-03	EM-2026-0001	Reject	Reject	q_trap	3/12	NO_TRANSITION	PASS
TC-INV-04	EMPP-2026-0001	Reject	Reject	q_trap	4/14	NO_TRANSITION	PASS
TC-INV-05	EMP2026-0001	Reject	Reject	q_trap	4/12	NO_TRANSITION	PASS
TC-INV-06	EMP-20260001	Reject	Reject	q_trap	9/12	NO_TRANSITION	PASS
TC-INV-07	EMP_2026_0001	Reject	Reject	—	0/13	INVALID_SYMBOL	PASS
TC-INV-08	EMP - -2026-0001	Reject	Reject	q_trap	5/14	NO_TRANSITION	PASS
TC-INV-09	EMP-26-0001	Reject	Reject	q_trap	7/11	NO_TRANSITION	PASS
TC-INV-10	EMP-20261-0001	Reject	Reject	q_trap	9/14	NO_TRANSITION	PASS
TC-INV-11	EMP-202A-0001	Reject	Reject	—	0/13	INVALID_SYMBOL	PASS
TC-INV-12	EMP-2026-1	Reject	Reject	q10	10/10	NON_FINAL_STATE	PASS
TC-INV-13	EMP-2026-00001	Reject	Reject	q_trap	14/14	NO_TRANSITION	PASS
TC-INV-14	EMP-2026-12B4	Reject	Reject	—	0/13	INVALID_SYMBOL	PASS
TC-INV-15	(empty)	Reject	Reject	—	0/0	EMPTY_INPUT	PASS
TC-INV-16	'EMP-2026-0001 '	Reject	Reject	—	0/14	INVALID_SYMBOL	PASS
TC-INV-17	' EMP-2026-0001 '	Reject	Reject	—	0/14	INVALID_SYMBOL	PASS
TC-INV-18	EMP-2026-00!1	Reject	Reject	—	0/13	INVALID_SYMBOL	PASS

Outcome omits the prefix REJECTED_. Symbols read is the number of symbols consumed by the automaton out of the input length; 0 means the input was rejected before the automaton started.

16.3 Sample Transition Traces

Table 20. Transition trace for the accepted input EMP-2026-0042

Step	Symbol	From	To	Step	Symbol	From	To
1	E	q0	q1	8	6	q7	q8
2	M	q1	q2	9	-	q8	q9
3	P	q2	q3	10	0	q9	q10
4	-	q3	q4	11	0	q10	q11
5	2	q4	q5	12	4	q11	q12
6	0	q5	q6	13	2	q12	q13

Step	Symbol	From	To	Step	Symbol	From	To
7	2	q6	q7				

Final state $q_{13} \in F$ after 13 of 13 symbols — ACCEPTED.

Two rejected runs illustrate the two automaton-level rejections. For EMP2026-0001 the automaton reaches q_3 after EMP, reads 2 where only - is allowed, and enters the dead state, so the run stops after 4 symbols: “Rejected at position 3: No transition: q_3 expects '-' but read '2'.” For EMP-2026-1 every symbol has a valid transition, but the input ends in q_{10} , which is not accepting: “Rejected: input ended in q_{10} , which is not accepting — at least 3 more symbol(s) needed (expected a digit 0–9).”

Table 21. Transition traces of two rejected inputs

Input	Step	Symbol	From	To	Result
EMP2026-0001	1	E	q_0	q_1	ok
	2	M	q_1	q_2	ok
	3	P	q_2	q_3	ok
	4	2	q_3	q_{trap}	dead
EMP-2026-1 (last 3 of 10 steps)	8	6	q_7	q_8	ok
	9	-	q_8	q_9	ok
	10	1	q_9	q_{10}	ok

16.4 Verification of Consistency between the Representations

Beyond individual examples, the project verifies that the regular expression and the three automata denote the same language. Table 22 summarizes these checks; the first four are also available in the Equivalence tab of the application (Figure 22).

Table 22. Consistency checks between RE, ϵ -NFA, DFA and minimized DFA

Claim	Method	Result
DFA $\equiv$ minimized DFA	product automaton, exhaustive breadth-first search	equal (15 state pairs, no distinguishing word)
Minimized DFA $\equiv$ hand-written reference DFA	product automaton	equal
RE $\equiv \epsilon$ -NFA $\equiv$ DFA $\equiv$ minimized DFA	every word over Σ of length ≤ 4 (41,371 words) compared with the oracle	0 disagreements (ϵ -NFA 0, DFA 0, minimized DFA 0)
All models agree near valid IDs	single-symbol deletions, insertions and substitutions of 40 random valid IDs	passed
All models agree on random input	20,000 random words, including Unicode look-alikes	passed
The check detects any error	each of the 2,940 single-transition mutants of the minimized DFA	every mutant detected with a distinguishing word

17 Screenshots of the Working System

The screenshots below were captured from the running application (window size 1240×800 at $2\times$ scale). They show the three views of the system processing real inputs; the numbers, states and messages are those produced by the program.

Table 23. Screens captured in this section

Figure	View	Model	Input	Shows
Figure 7	Simulate	Minimal DFA	EMP - 2026 - 0042	input loaded, start state
Figure 8	Simulate	Minimal DFA	EMP - 2026 - 0042	run in progress (step 6)
Figure 9	Simulate	Minimal DFA	EMP - 2026 - 0042	ACCEPTED
Figure 10	Simulate	Minimal DFA	EMP2026 - 0001	REJECTED – dead state
Figure 11	Simulate	Minimal DFA	EMP - 2026 - 12A4	REJECTED – symbol not in Σ
Figure 12	Simulate	Minimal DFA	EMP - 2026 - 1	REJECTED – non-accepting state
Figure 13	Simulate	DFA before min.	EMP - 2026 - 0042	subset states during a run
Figure 14	Simulate	ϵ -NFA	EMP - 2026 - 0042	set of active NFA states
Figure 15	Theory	—	—	Language tab
Figure 16	Theory	—	—	Regular expression tab
Figure 17	Theory	—	—	ϵ -NFA tab
Figure 18	Theory	—	—	Subset construction tab
Figure 19	Theory	—	—	DFA tab
Figure 20	Theory	—	—	Minimization tab
Figure 21	Theory	—	EMP - 2026 - 0042	Minimal DFA tab with last run
Figure 22	Theory	—	all words, $ w \leq 4$	Equivalence tab
Figure 23	Tests	all three	28 predefined cases	test suite results
Figure 24	Tests	Minimal DFA	8 IDs	batch validation

17.1 Simulate View

Employee ID Validator EMP-YYYY-NNNN · RE → NFA → DFA → minimal DFA

Simulate Theory Tests

EMP - 2026 - 0042 | Load | Examples... | Model: Minimal DFA (authoritative)

Regular-expression parts: ■ prefix EMP ■ separator - ■ year YYYY D⁴ ■ separator - ■ number NNNN D⁴

Symbol: — State: q0

READY Layer 1 passed: all 13 symbols are in Σ . The automaton is at its start state — press Step or Play.

Minimal DFA (authoritative recognizer)

Transition history

Double ring = accepting · amber = current · tinted = visited · dashed red = dead. Hover or click a state or transition to explain it; scroll to zoom, drag to pan.

Prev Play Step Run all Reset

Speed 1.0x

Figure 7. Simulate view after loading EMP - 2026 - 0042: Layer 1 has accepted all 13 symbols and the minimal DFA waits in the start state q0.

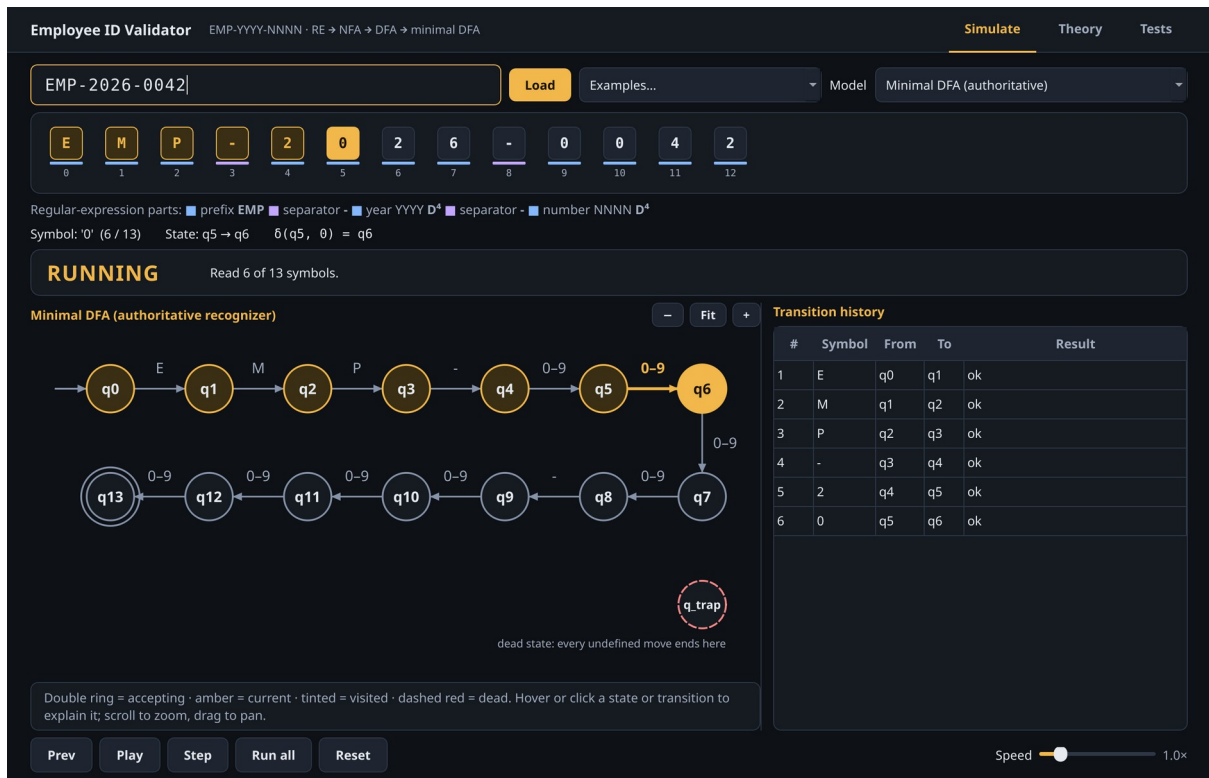


Figure 8. Step-by-step run after six symbols: the current state q6 is highlighted, visited states are tinted and the history lists the transitions taken.

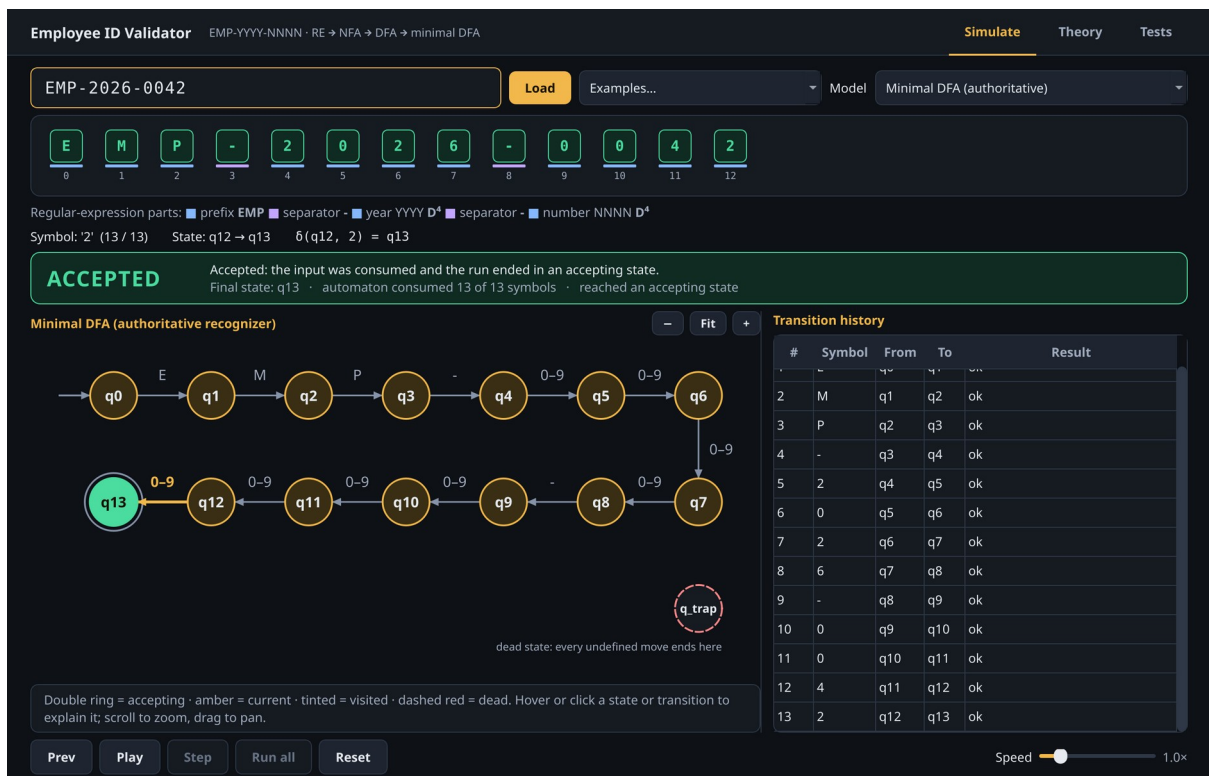


Figure 9. Accepted input EMP-2026-0042: all 13 symbols consumed, final state q13 ∈ F.

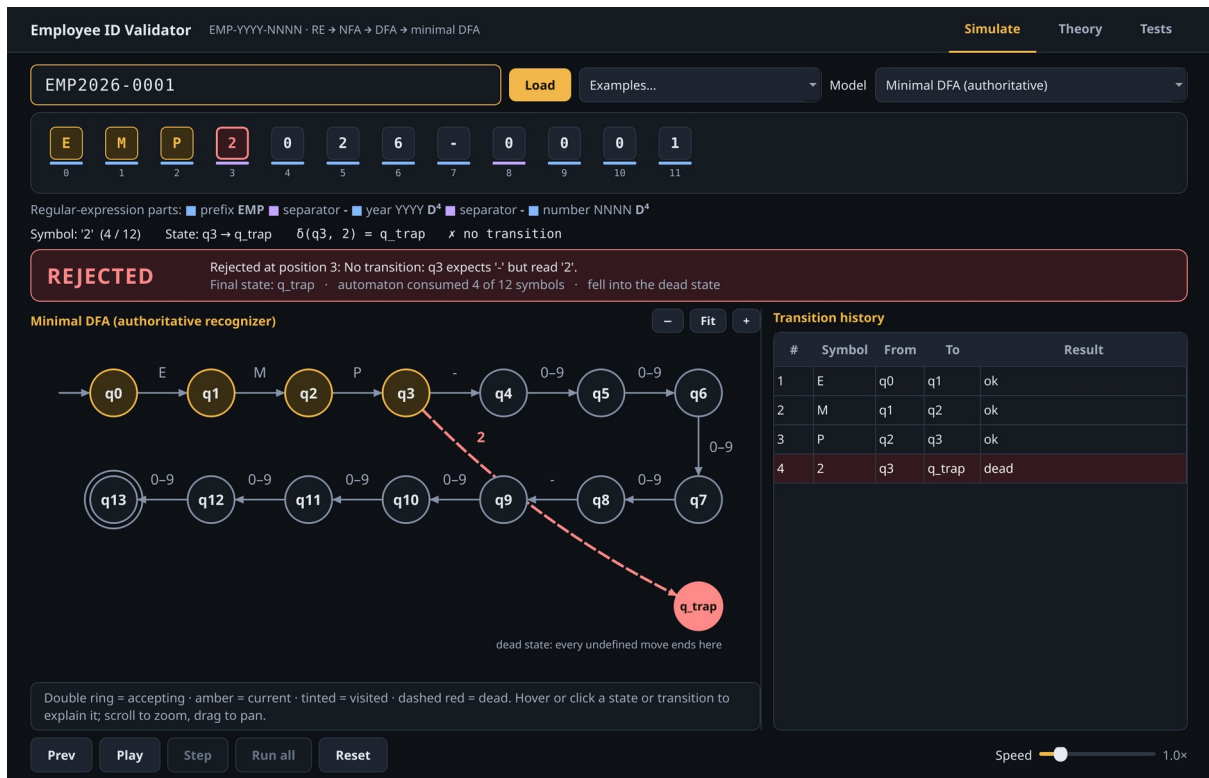


Figure 10. Rejected input EMP2026-0001: at position 3 the automaton in q3 expects '-' but reads '2' and enters q_trap.

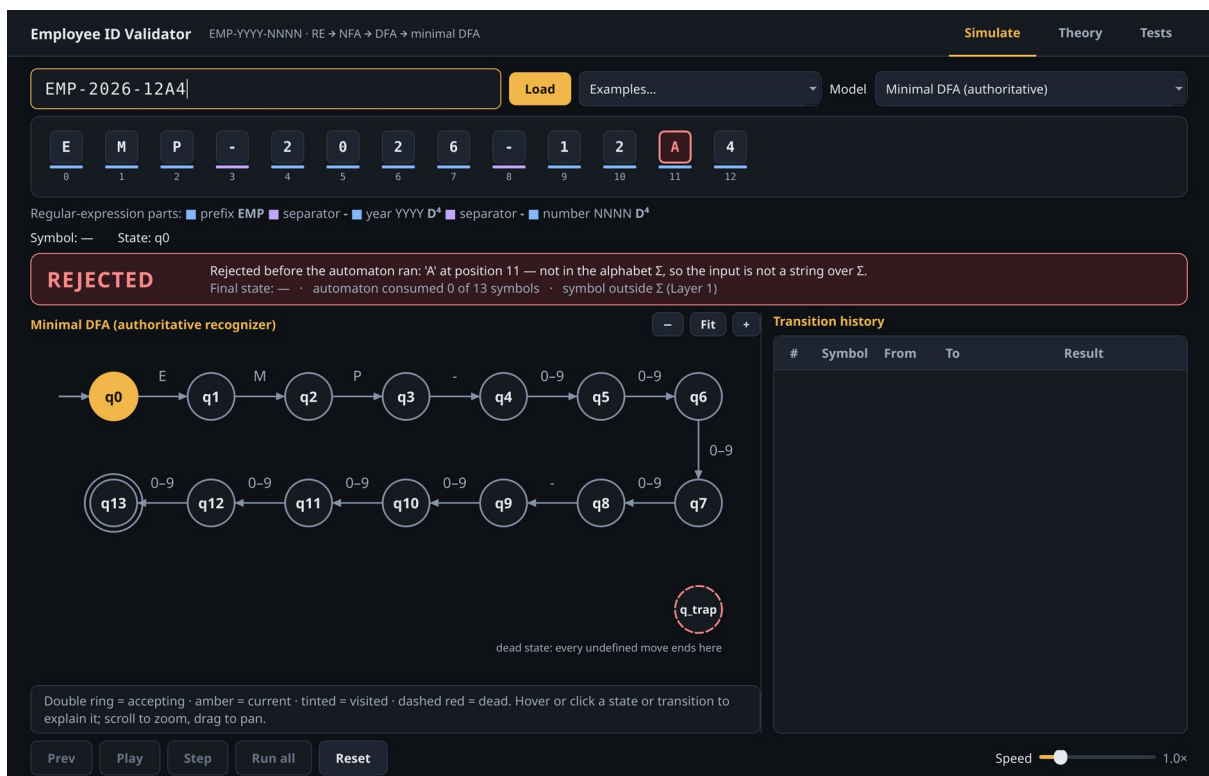


Figure 11. Rejected input EMP-2026-12A4: 'A' at position 11 is not in Σ, so the input is rejected by Layer 1 before the automaton runs.

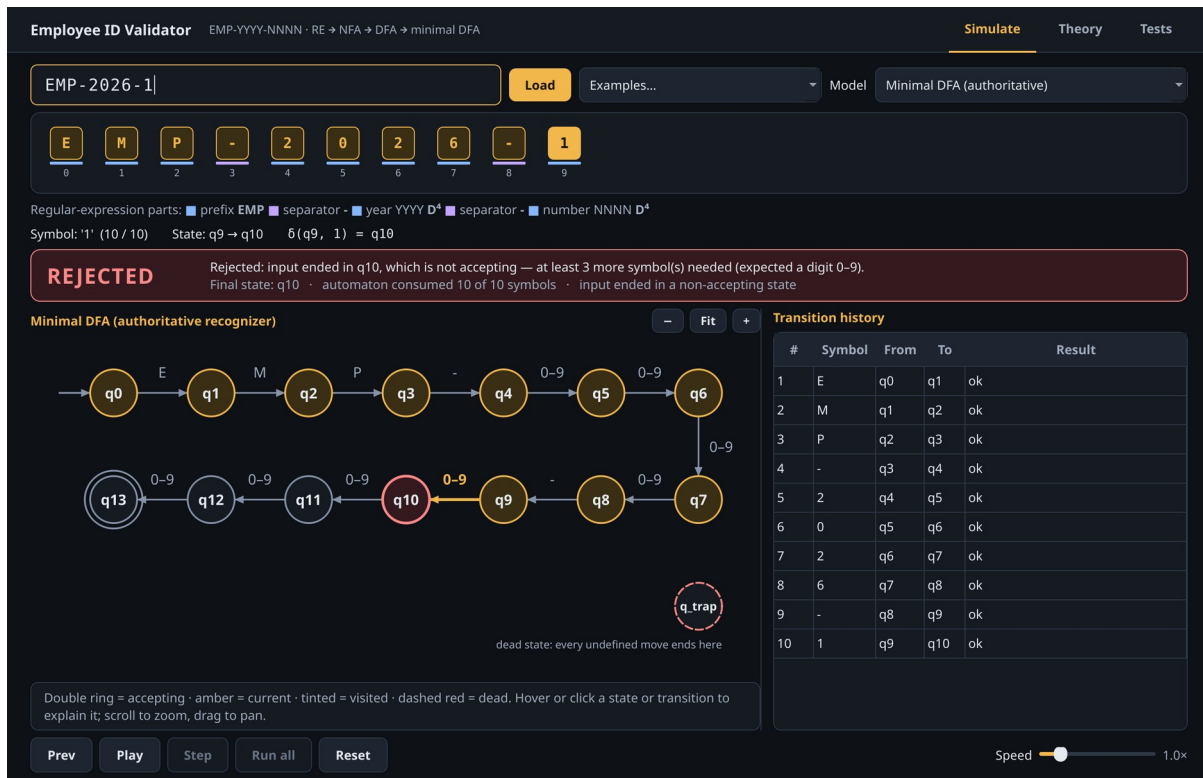


Figure 12. Rejected input EMP-2026-1: every transition is valid, but the input ends in the non-accepting state q10; three more symbols would be needed.

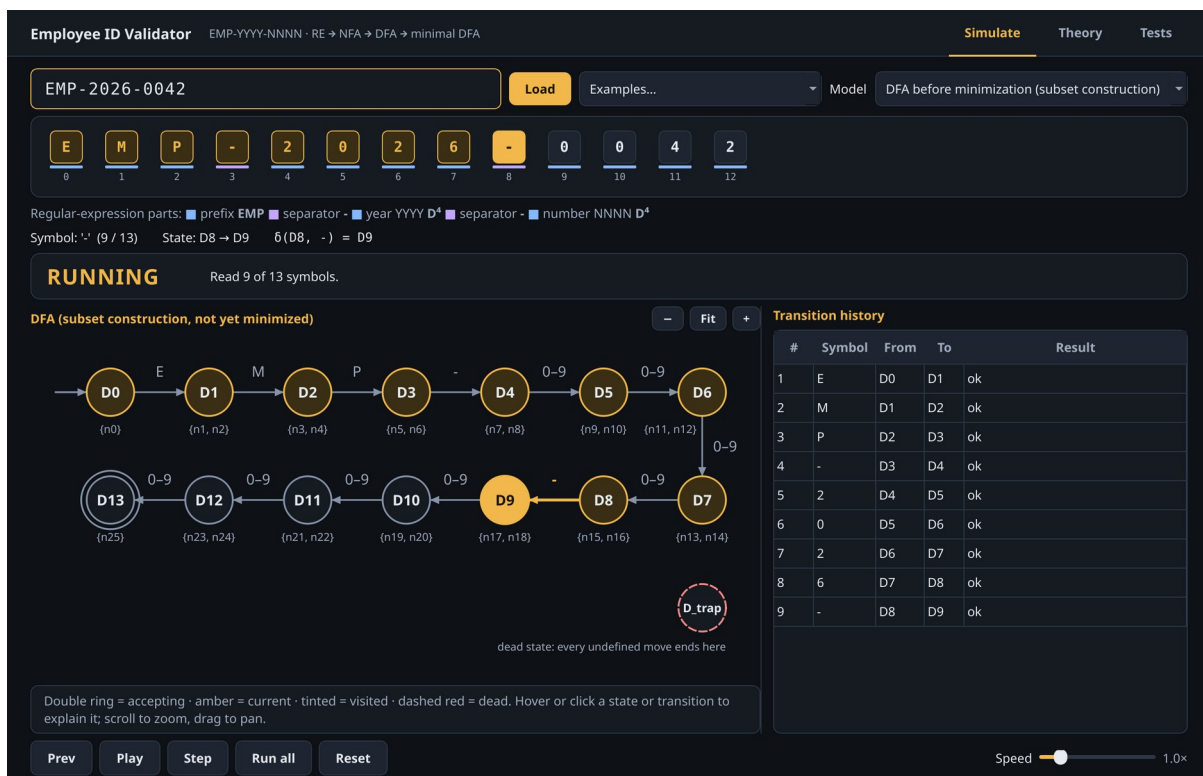


Figure 13. The same input run on the DFA before minimization; each state is shown with the NFA subset it represents.

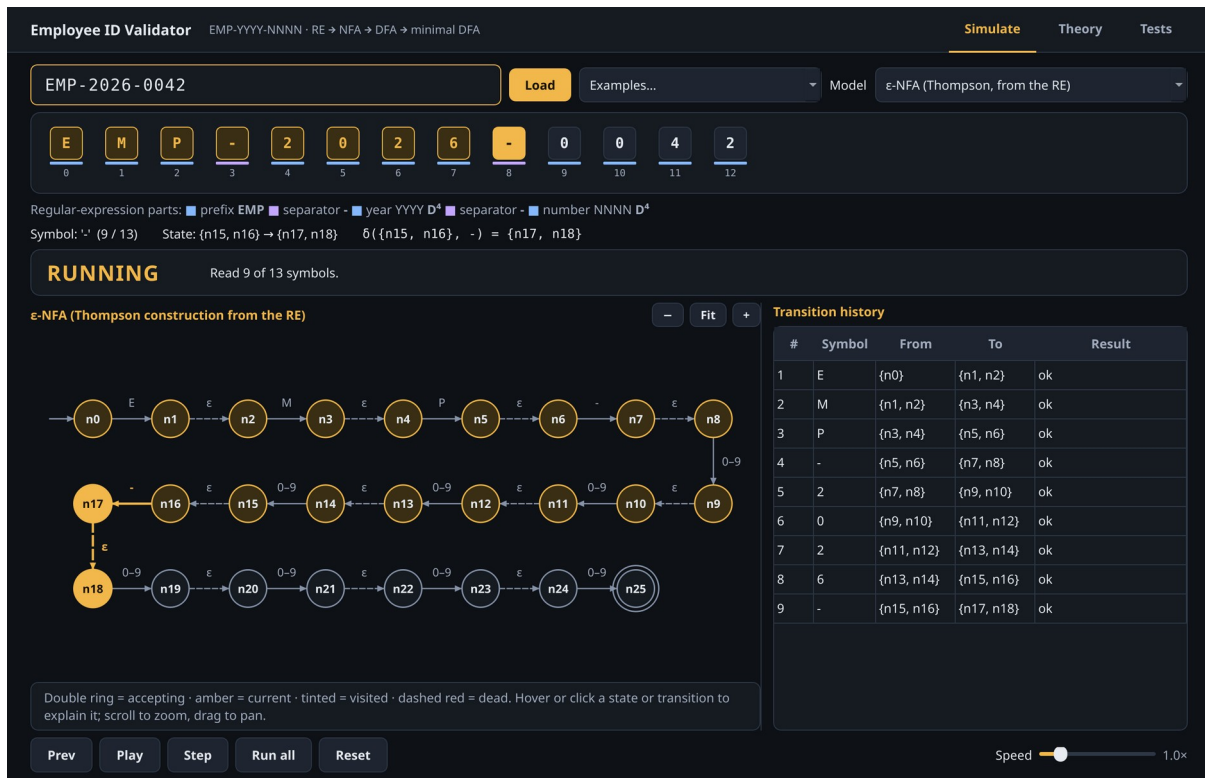


Figure 14. The same input run on the ϵ -NFA: the current state is a set of NFA states, {n17, n18}, after nine symbols.

17.2 Theory View

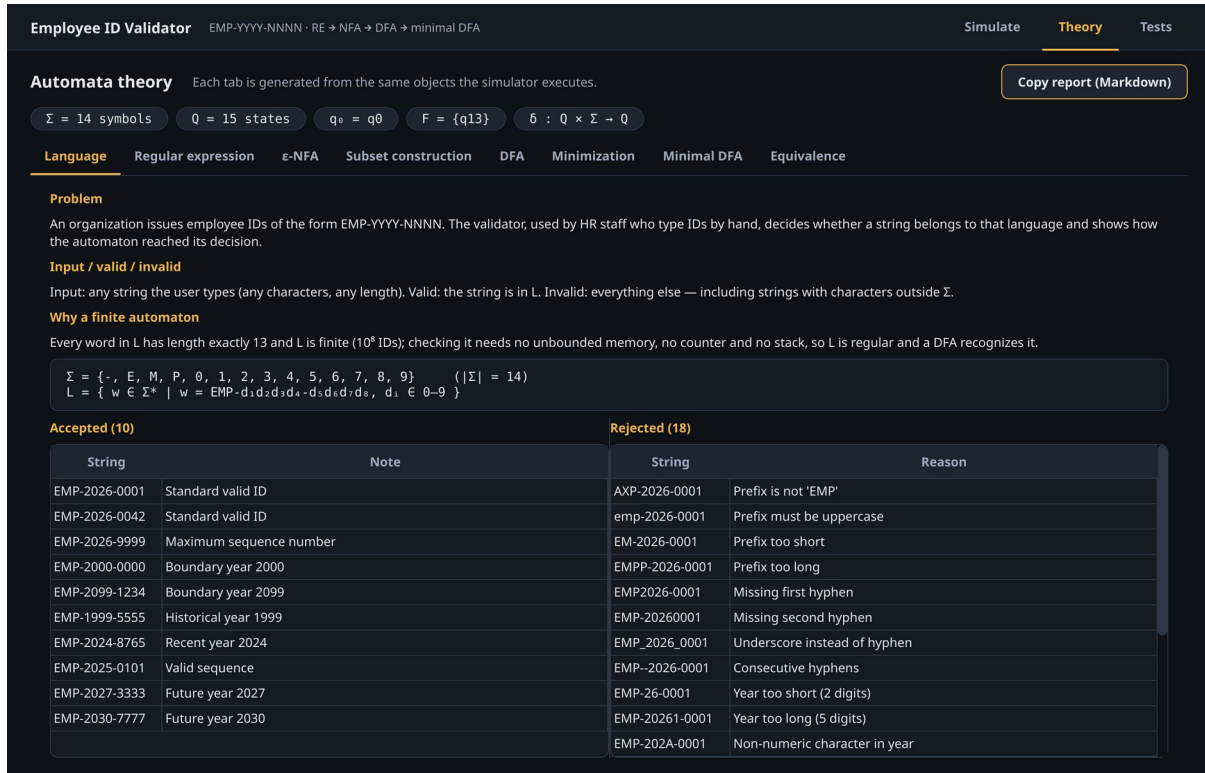


Figure 15. Theory view, Language tab: problem definition, Σ , L and the accepted and rejected examples.

Employee ID Validator EMP-YYYY-NNNN · RE → NFA → DFA → minimal DFA Simulate **Theory** Tests

Automata theory Each tab is generated from the same objects the simulator executes. Copy report (Markdown)

$\Sigma = 14$ symbols $Q = 15$ states $q_0 = q_0$ $F = \{q_{13}\}$ $\delta : Q \times \Sigma \rightarrow Q$

Language **Regular expression** ϵ -NFA Subset construction DFA Minimization Minimal DFA Equivalence

Formal regular expression

EMP-D⁴-D⁴

E M P - 2 0 2 6 - 0 0 4 2

Regular-expression parts: ■ prefix EMP ■ separator - ■ year YYYY D⁴ ■ separator - ■ number NNNN D⁴

E·M·P·-·D·D·D·D·-·D·D·D·D where D = {0u1u2u3u4u5u6u7u8u9}

Union (u), concatenation (·) and bounded repetition (superscript) are the only operators. Python check pattern (test oracle only, never used for acceptance): EMP-[0-9]{4}-[0-9]{4} with re.fullmatch.

Components

Position	Component	Meaning	Example
1-3	EMP	fixed uppercase prefix E-M-P	EMP
4	-	first separator	-
5-8	D ⁴	exactly four digits: the year YYYY (any four digits — syntax only)	2026
9	-	second separator	-
10-13	D ⁴	exactly four digits: the employee number NNNN	0042

Why these strings do not match

String	Reason
AXP-2026-0001	Prefix is not 'EMP'

Figure 16. Theory view, Regular expression tab: the formal expression, its coloured segments and its components.

Employee ID Validator EMP-YYYY-NNNN · RE → NFA → DFA → minimal DFA Simulate **Theory** Tests

Automata theory Each tab is generated from the same objects the simulator executes. Copy report (Markdown)

$\Sigma = 14$ symbols $Q = 15$ states $q_0 = q_0$ $F = \{q_{13}\}$ $\delta : Q \times \Sigma \rightarrow Q$

Language **Regular expression** **ϵ -NFA** Subset construction DFA Minimization Minimal DFA Equivalence

M NFA = (Q, Σ , δ , q_0 , F)
 $Q = \{n_0, n_1, \dots, n_{25}\}$ (26 states)
 $\Sigma = \{E, M, P, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (14 symbols)
 $q_0 = n_0$
 $F = \{n_{25}\}$

Thompson construction: each of the 13 symbols becomes an atom of two states (a digit atom has 10 parallel edges); 12 ϵ -edges glue the atoms together. → marks the start state, * the accepting state. A missing move means $\emptyset$ — an NFA has no trap state.

State	E	M	P	-	0-9	ϵ
→ n0	{n1}	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
n1	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n2}
n2	$\emptyset$	{n3}	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
n3	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n4}
n4	$\emptyset$	$\emptyset$	{n5}	$\emptyset$	$\emptyset$	$\emptyset$
n5	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n6}
n6	$\emptyset$	$\emptyset$	$\emptyset$	{n7}	$\emptyset$	$\emptyset$
n7	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n8}
n8	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n9}
n9	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n10}
n10	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n11}
n11	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n12}
n12	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	{n13}	$\emptyset$

ϵ -NFA (Thompson construction from the RE) Fit +

Double ring = accepting · amber = current · tinted = visited · dashed red = dead. Hover or click a state or transition to explain it; scroll to zoom, drag to pan.

Figure 17. Theory view, ϵ -NFA tab: five-tuple, transition table and the interactive state diagram.

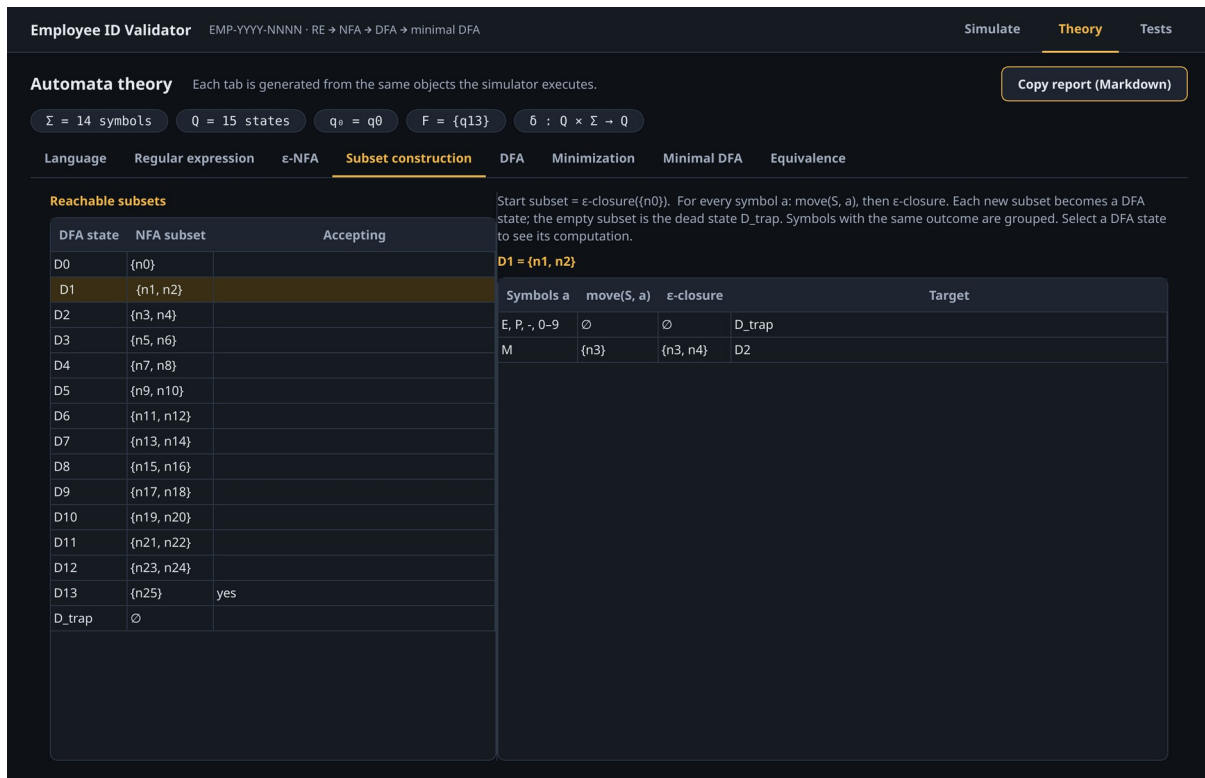


Figure 18. Theory view, Subset construction tab: the reachable subsets and the move / ϵ -closure computed for the selected state.

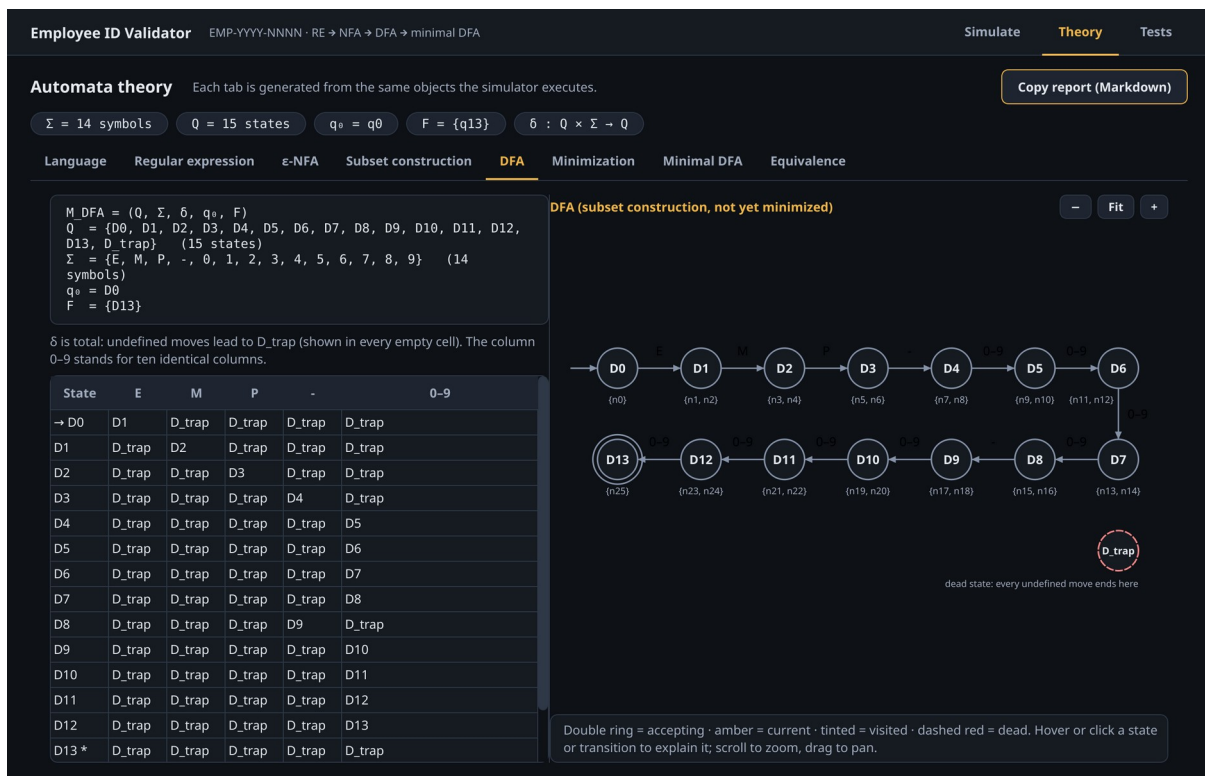


Figure 19. Theory view, DFA tab: the DFA before minimization with its transition table and diagram.

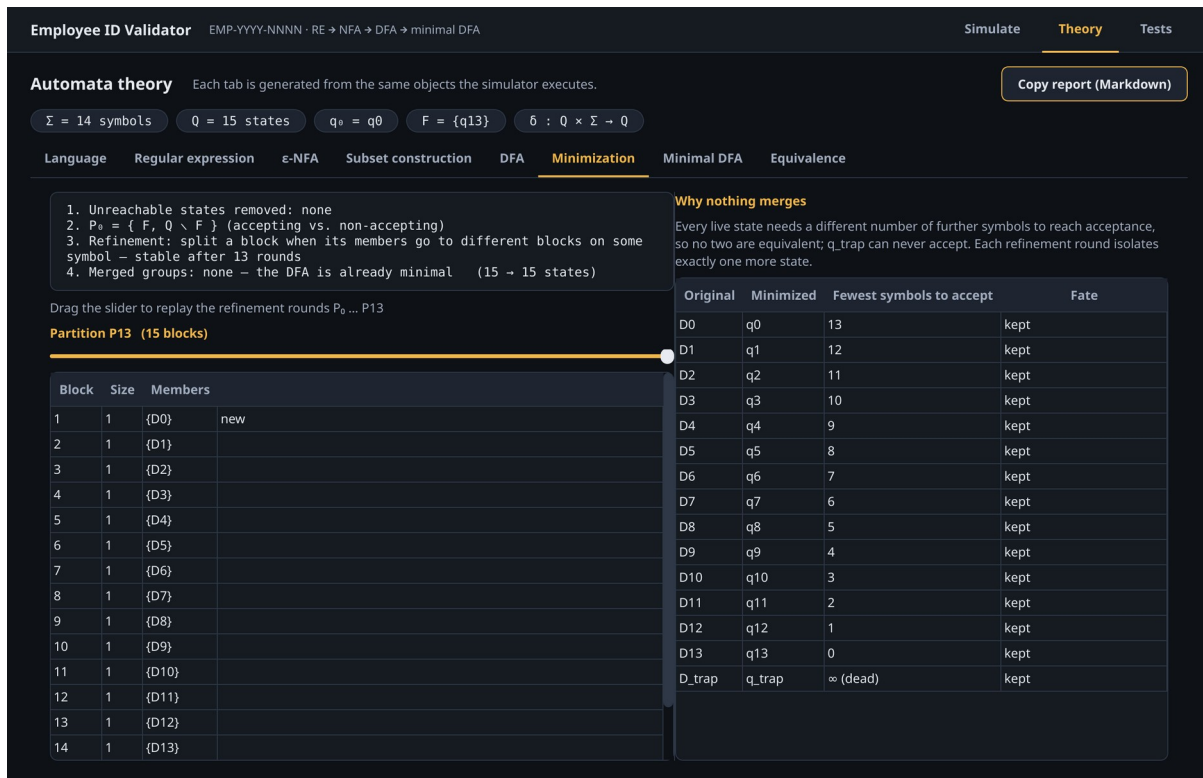


Figure 20. Theory view, Minimization tab: summary of the procedure, the partition after the last round and the distinguishing measure of each state.

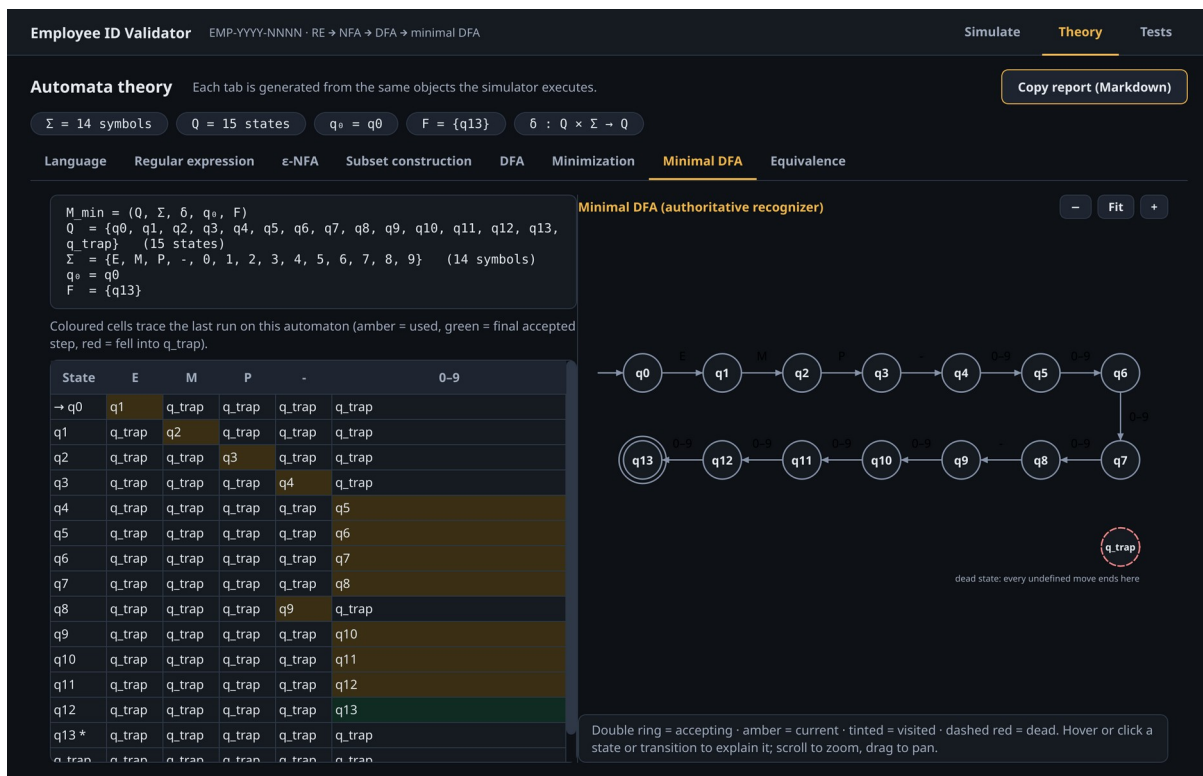


Figure 21. Theory view, Minimal DFA tab: the path of the last run (EMP-2026-0042) is coloured in the transition table.

Employee ID Validator EMP-YYYY-NNNN · RE → NFA → DFA → minimal DFA Simulate **Theory** Tests

Automata theory Each tab is generated from the same objects the simulator executes. Copy report (Markdown)

$\Sigma = 14$ symbols $Q = 15$ states $q_0 = q_0$ $F = \{q_{13}\}$ $\delta : Q \times \Sigma \rightarrow Q$

Language Regular expression ϵ -NFA Subset construction DFA Minimization Minimal DFA **Equivalence**

Do RE, NFA, DFA and minimal DFA recognize the same language?

Claim	Method	Result
Subset DFA $\equiv$ minimal DFA	Product automaton, exhaustive BFS	EQUAL — 15 state pairs explored, no distinguishing word
Minimal DFA $\equiv$ hand-written reference table	Product automaton (reference is used only for this check)	EQUAL — 15 state pairs explored, no distinguishing word
ϵ -NFA $\equiv$ DFA	Subset construction preserves the language; sampled exhaustively below	EQUAL on all 41,371 words
RE $\equiv$ ϵ -NFA	Thompson construction preserves the language; PCRE oracle below	EQUAL on all 41,371 words

Exhaustive check: every word over Σ up to length 4 (41,371 words) is run through the NFA, the subset DFA, the minimal DFA and the PCRE oracle; any disagreement is counted.

Run exhaustive check

41,371 words checked. Disagreements with the oracle — ϵ -NFA: 0, subset DFA: 0, minimal DFA: 0

Figure 22. Theory view, Equivalence tab after running the exhaustive check: 41,371 words, no disagreement.

17.3 Tests View

Employee ID Validator EMP-YYYY-NNNN · RE → NFA → DFA → minimal DFA Simulate Theory **Tests**

Test suite (28) Batch

Run all 28 / 28 passed · NFA, DFA and minimal DFA agree on 28 / 28 inputs

Test ID	Category	Input	Expected	Actual	Result	Models agree	Reason	
TC-VAL-01	Valid Standard	EMP-2026-0001	ACCEPTED	ACCEPTED	PASS	yes	Standard valid ID	Simulate
TC-VAL-02	Valid Standard	EMP-2026-0042	ACCEPTED	ACCEPTED	PASS	yes	Standard valid ID	Simulate
TC-VAL-03	Valid Standard	EMP-2026-9999	ACCEPTED	ACCEPTED	PASS	yes	Maximum sequence number	Simulate
TC-VAL-04	Valid Boundary	EMP-2000-0000	ACCEPTED	ACCEPTED	PASS	yes	Boundary year 2000	Simulate
TC-VAL-05	Valid Boundary	EMP-2099-1234	ACCEPTED	ACCEPTED	PASS	yes	Boundary year 2099	Simulate
TC-VAL-06	Valid Standard	EMP-1999-5555	ACCEPTED	ACCEPTED	PASS	yes	Historical year 1999	Simulate
TC-VAL-07	Valid Standard	EMP-2024-8765	ACCEPTED	ACCEPTED	PASS	yes	Recent year 2024	Simulate
TC-VAL-08	Valid Standard	EMP-2025-0101	ACCEPTED	ACCEPTED	PASS	yes	Valid sequence	Simulate
TC-VAL-09	Valid Future	EMP-2027-3333	ACCEPTED	ACCEPTED	PASS	yes	Future year 2027	Simulate
TC-VAL-10	Valid Future	EMP-2030-7777	ACCEPTED	ACCEPTED	PASS	yes	Future year 2030	Simulate
TC-INV-01	Invalid Prefix	AXP-2026-0001	REJECTED	REJECTED	PASS	yes	Prefix is not 'EMP'	Simulate
TC-INV-02	Invalid Prefix	emp-2026-0001	REJECTED	REJECTED	PASS	yes	Prefix must be uppercase	Simulate
TC-INV-03	Invalid Prefix	EM-2026-0001	REJECTED	REJECTED	PASS	yes	Prefix too short	Simulate
TC-INV-04	Invalid Prefix	EMPP-2026-0001	REJECTED	REJECTED	PASS	yes	Prefix too long	Simulate
TC-INV-05	Missing Separator	EMP2026-0001	REJECTED	REJECTED	PASS	yes	Missing first hyphen	Simulate
TC-INV-06	Missing Separator	EMP-20260001	REJECTED	REJECTED	PASS	yes	Missing second hyphen	Simulate
TC-INV-07	Wrong Separator	EMP_2026_0001	REJECTED	REJECTED	PASS	yes	Underscore instead of hyphen	Simulate
TC-INV-08	Extra Separator	EMP--2026-0001	REJECTED	REJECTED	PASS	yes	Consecutive hyphens	Simulate
TC-INV-09	Invalid Year	EMP-26-0001	REJECTED	REJECTED	PASS	yes	Year too short (2 digits)	Simulate
TC-INV-10	Invalid Year	EMP-20261-0001	REJECTED	REJECTED	PASS	yes	Year too long (5 digits)	Simulate

Figure 23. Tests view: 28 / 28 predefined cases passed, and the NFA, DFA and minimal DFA agree on all 28 inputs.

Employee ID Validator EMP-YYYY-NNNN · RE → NFA → DFA → minimal DFA Simulate Theory Tests

Test suite (28) **Batch**

EMP-2026-0042
EMP-1999-5555
EMP-26-0001
EMP-2026-12A4
emp-2026-0001
EMP-2026-1
EMP-2026-0001
EMP-2030-7777

Validate all 8 inputs · 3 accepted · 5 rejected

#	Input	Verdict	Final state	Reason
1	EMP-2026-0042	ACCEPTED	q13	Accepted: the input was consumed and the run ended in an accepting state.
2	EMP-1999-5555	ACCEPTED	q13	Accepted: the input was consumed and the run ended in an accepting state.
3	EMP-26-0001	REJECTED	q_trap	Rejected at position 6: No transition: q6 expects a digit 0–9 but read '·'.
4	EMP-2026-12A4	REJECTED	—	Rejected before the automaton ran: 'A' at position 11 — not in the alphabet Σ , so the input is not a string over Σ .
5	emp-2026-0001	REJECTED	—	Rejected before the automaton ran: 'e' at position 0, 'm' at position 1, 'p' at position 2 — not in the alphabet Σ , so the input is not a string ...
6	EMP-2026-1	REJECTED	q10	Rejected: input ended in q10, which is not accepting — at least 3 more symbol(s) needed (expected a digit 0–9).
7	EMP-2026-0001	REJECTED	q_trap	Rejected at position 4: No transition: q4 expects a digit 0–9 but read '·'.
8	EMP-2030-7777	ACCEPTED	q13	Accepted: the input was consumed and the run ended in an accepting state.

Figure 24. Tests view, Batch tab: eight IDs validated at once, with verdict, final state and reason for each.

18 Discussion of Results

18.1 Correctness of the Recognizer

All 28 predefined cases produced the expected verdict, and the three automata agreed on each of them. More importantly, correctness does not rest on these examples alone. The equality of the DFA and the minimized DFA was *proved* by the product-automaton search, and the minimized DFA was shown to equal an independently hand-written reference table. Agreement of all representations with the regular expression was established exhaustively for every one of the 41,371 words of length at most four, and by 20,000 random words and the systematic single-symbol variations of valid IDs. The mutation test shows that the equivalence check is sensitive: redirecting any single one of the 210 transitions of the minimized DFA to any other state changes the language, and all 2,940 such mutants were detected.

18.2 Distribution of Rejection Outcomes

The 18 rejected cases were rejected for different reasons, and the outcome reported by the system matches the kind of error. Eight were rejected at Layer 1 ($8 \times \text{INVALID_SYMBOL}$) because they contain a symbol outside Σ ; eight were rejected by the automaton entering the dead state ($8 \times \text{NO_TRANSITION}$); one ended in a non-accepting state (EMP-2026-1) and one was empty.

Two results deserve comment. The prefix tests AXP-2026-0001 and emp-2026-0001 were designed as structural errors, but they are rejected by the alphabet check, not by the automaton, because A, X and the lowercase letters are not symbols of Σ . This is the correct behaviour — such strings are not words over Σ , so δ is not defined on them — but it means that the dead-state path for a wrong prefix is exercised by EMP-2026-0001 and EMP-2026-0001 instead. Second, a wrong length is detected in two different ways depending on its direction: an extra symbol (EMP-2026-00001, EMP-20261-0001) leads to q_trap at the first surplus position, while a missing symbol at the end (EMP-2026-1) leaves the run in

a non-accepting state. These are exactly the two ways in which a word can fail to belong to a fixed-length language, as predicted in Section 4.3.

18.3 Consistency between the Formal Material and the Implementation

The DFA produced by subset construction turned out to be already minimal: minimization merged no states and only renamed them ($15 \rightarrow 15$). This is consistent with the structure of L : the minimal DFA of a language has one state per class of the Myhill–Nerode equivalence, and for this fixed-length language the classes are exactly “ i symbols of a valid ID matched” for $i = 0, \dots, 13$ plus the class of invalid prefixes — fifteen in total. Because the theory tables in this document, in the docs/ folder and in the Theory view are all generated from the same objects that the simulator runs, the formal definition, the regular expression, the automata and the implementation cannot drift apart; a test fails if any documentation table differs from the code.

18.4 Observations on the System

- The ϵ -NFA is larger than the DFA (26 states against 15) because Thompson's construction introduces a two-state fragment per atom and ϵ -edges between them. The ϵ -transitions are what make the ϵ -closure computation visible in the subset-construction trace.
- Rejecting strings outside Σ before running the automaton gives clearer feedback than letting them fail inside it: the user is told every offending character and its position at once, and the explanation distinguishes “not a symbol of the alphabet” from “a valid symbol in the wrong place”.
- Stopping the run as soon as the dead state is entered pinpoints the first incorrect position, which is the information a person correcting a typed ID needs.
- The validator is purely syntactic. Strings such as EMP-0000-0000 are accepted because the year field is defined as any four digits; a semantic check, for example of a valid year range, would require either a larger automaton or a separate rule, and was outside the scope of the project.

19 Conclusion

The project met its objectives. The organization-defined employee-ID format EMP-YYYY-NNNN was defined as a finite, and therefore regular, language L over a 14-symbol alphabet, and described by the regular expression $\text{EMP-D}^4\text{-D}^4$. From this one definition the system derives, by Thompson's construction, an ϵ -NFA with 26 states and 12 ϵ -transitions; by subset construction, a complete DFA with 15 states including the dead state; and by Moore's partition refinement, the minimized DFA, which after thirteen refinement rounds was found to have no equivalent states and therefore also 15 states, q_0 to q_{13} and q_{trap} .

The minimized DFA is the authoritative recognizer of a PySide6 application that validates IDs symbol by symbol, highlights the current state, records every transition and explains each rejection by its position and cause, while presenting the complete derivation in its Theory view. All 28 predefined test cases gave the expected result, all 279 automated tests passed, and the consistency of the regular expression, the three automata and the implementation was established by an exact product-automaton proof, an exhaustive comparison of 41,371 words, randomized testing and mutation testing.

The work shows in practice the equivalence of regular expressions, NFAs and DFAs that is central to the course, and how minimization yields a unique smallest recognizer. Possible extensions are the

support of several ID formats, a semantic check of the year field, and a regular-expression parser that would allow the format to be changed from the interface rather than in the source code.

20 References

- Graphviz. (n.d.). *Graphviz: Graph visualization software*. Retrieved October 1, 2026, from <https://graphviz.org/>
- Hopcroft, J. (1971). An $n \log n$ algorithm for minimizing states in a finite automaton. In Z. Kohavi & A. Paz (Eds.), *Theory of machines and computations* (pp. 189–196). Academic Press.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson Addison-Wesley.
- Linz, P. (2017). *An introduction to formal languages and automata* (6th ed.). Jones & Bartlett Learning.
- Moore, E. F. (1956). Gedanken-experiments on sequential machines. In C. E. Shannon & J. McCarthy (Eds.), *Automata studies* (pp. 129–153). Princeton University Press.
- pytest-dev. (n.d.). *pytest documentation*. Retrieved October 1, 2026, from <https://docs.pytest.org/>
- Python Software Foundation. (n.d.). *re — Regular expression operations*. Python 3 documentation. Retrieved October 1, 2026, from <https://docs.python.org/3/library/re.html>
- Rabin, M. O., & Scott, D. (1959). Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2), 114–125. <https://doi.org/10.1147/rd.32.0114>
- Sipser, M. (2013). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.
- The Qt Company. (n.d.). *Qt for Python (PySide6) documentation*. Retrieved October 1, 2026, from <https://doc.qt.io/qtforpython-6/>
- Thompson, K. (1968). Programming techniques: Regular expression search algorithm. *Communications of the ACM*, 11(6), 419–422. <https://doi.org/10.1145/363347.363387>

21 Individual Contribution Matrix

Table 24 records the role and the specific contributions of each member of the group.

Table 24. Individual contribution matrix

Member	Role	Contributions
Anub, Mark Christian T.		
Ayson, Rey Noel C.		
Dotosme, James Paul B.		
Lauzon, Chester Josh C.		
Perito, Isiah Thomas A.		
Talingting, Ken Ira L.		